BVCC Wallet

BLOCK VENTURE CHAIN CAPITAL

Agent Wallet Informe de Seguridad y Pruebas

Cinco generaciones, de V1 a V5: rondas de revisión, remediación, despliegue y verificación on-chain

OBJETO	BVCC Agent Wallet (smart account ERC-4337 / ERC-7579)
CÓDIGO	**V5 (actual):** commit `838a70f` · árbol de `contracts-v5/src` `64a5c674` **V4 (anterior):** commit `a3e28f1` · árbol de `contracts/src` `337e84f0`
DESPLIEGUE	Arbitrum One · Ethereum · BNB Chain · Base · Polygon · Arbitrum Sepolia
TRABAJO	Revisión de seguridad **interna**; las rondas de V5, con herramientas de análisis con IA. No es una auditoría externa
PERIODO	2026-06-06 → 2026-11-11 · de V1 a V4, después cinco rondas de V5 y un ejercicio de equipo rojo
HALLAZGOS	2 Críticos · 13 Altos · 10 Medios · 3 Bajos · 2 Informativos
ESTADO (V5)	26 cerrados · 4 aceptados · 0 abiertos · 191/191 tests · 72/72 contratos verificados

Índice

PARTE I — AUDITORÍA		
1	Resumen ejecutivo	Resultados principales y qué debe llevarse un lector
2	Alcance, metodología y modelo de severidad	Qué se revisó, en qué commit, cómo se asigna la severidad
3	Registro de hallazgos	Todos los hallazgos, su severidad y su estado, en una página
4	Hallazgos en detalle	BVCC-01 … BVCC-13 (revisión de V4) · BVCC-14 … BVCC-30 (revisión de V5)
5	Remediación y verificación del despliegue	V5 y V4: qué está vivo, en qué redes y cómo se comprobó
6	Riesgo residual y recomendaciones	Qué sigue expuesto y qué hacer al respecto
7	Conclusión y limitaciones	Valoración, y qué no establece esta revisión

PARTE II — ANEXOS		
A	Arquitectura de seguridad	Primero el diseño de V5; después call policies y registry de V3/V4
B	Evidencia de pruebas	Suite de V5 y pruebas en cadena; baterías de V1 a V4
C	Análisis estático	Ejecuciones de Slither y registro de falsos positivos
D	Previamente remediado — V1 y V2	Hallazgos cerrados en generaciones superadas, con su evidencia
E	Cronología de la remediación	Todas las rondas, de V1 a V5, comprimidas
F	Identificadores y direcciones	Contratos, redes, build
G	Notas de proceso	En qué se equivocó esta revisión y qué implica

Los hallazgos se citan por identificador (`BVCC-01`) a lo largo de todo el documento, no por número de sección. Las secciones van por número y los anexos por letra.

1 Resumen ejecutivo

La BVCC Agent Wallet es una smart account ERC-4337 que permite a su dueño delegar en un agente autónomo (una EOA) una capacidad de gasto acotada. Este informe recoge todas las revisiones de esa vía de delegación, desde la primera batería de permisos de junio de 2026 hasta la generación V5, desplegada el 2026-11-02. La V5 es la generación actual. De V1 a V4 siguen en el informe porque las wallets creadas por sus factories siguen existiendo y conservan su propio bytecode.

30	2	0	191
HALLAZGOS, REVISIONES DE V4 Y V5	CRÍTICOS	ABIERTOS EN V5	TESTS DE V5, TODOS PASAN

V5, la generación actual

La V5 reescribe el modelo de permisos del agente en lugar de parchearlo. La revisión de V4 dejó abierto un riesgo material, **BVCC-03**: el presupuesto de token de un agente no acotaba lo que podía gastar a través de un protocolo al que el dueño ya hubiera dado una allowance. La V5 lo cierra midiendo lo que sale de la wallet. Cada lote de un agente lee el saldo de cada token relevante antes y después de ejecutarse y carga la diferencia, comisión incluida, contra los límites del agente. Da igual cómo se combinen las llamadas o qué allowances existan ya. Toda llamada de un agente a un protocolo necesita ahora un validator on-chain. Un límite de 0 significa que no se permite nada y una lista de destinatarios vacía significa ningún destinatario. Los permisos de un agente caducan en un año como mucho, las firmas del dueño llevan caducidad, y los guardianes votan ahora una propuesta concreta, lo que cierra el bloqueo de recuperación de **BVCC-09**.

Antes de desplegarse, el código de V5 pasó por cinco rondas de revisión y un ejercicio de equipo rojo. Entre todos encontraron **diecisiete problemas**, de BVCC-14 a BVCC-30: diez altos, cinco medios y dos bajos. Ninguno llegó a una red. Trece se corrigieron en el código. Cuatro se aceptan y se documentan como supuestos de confianza: los validators deben ser inmutables, los hooks de Uniswap v4 aprobados y los tokens permitidos son de confianza, y la wallet solo mide los tokens que declara un validator. La quinta ronda no encontró nada nuevo y el código se congeló el 2026-10-29. La mayoría de los altos están en los validators de posiciones de Uniswap, y varios aparecieron justo al lado del arreglo del anterior; el Anexo G vuelve sobre ello.

La V5 está viva en Ethereum, Arbitrum One, Base, BNB Chain, Polygon y Arbitrum Sepolia desde el 2026-11-02. Los 72 contratos (12 por red) tienen el código fuente verificado en los exploradores, y los 30 validators están activos en el registry. Ambas cosas se releyeron de cada red para esta edición, el 2026-11-11.

V4 y anteriores: qué encontró la revisión

Dos hallazgos son críticos. **BVCC-01**, una reentrada entre funciones, permitía a un agente cuya dirección adquiriera código después volver a entrar en el `execute()` de nivel de dueño y saltarse a la vez todos los límites configurados; una prueba de concepto movió 49,925 ETH con un presupuesto total de 0,003 ETH, y el mismo ataque funcionó contra una wallet creada desde la factory V2 viva en un fork de mainnet. **BVCC-02**, una máscara de comandos errónea en el validator del Universal Router, habría aceptado un depósito de puente entre cadenas como si fuera un swap —justo la clase

de exfiltración que el validator existe para impedir— y se detectó en la revisión antes de que ese validator se registrara en ninguna red.

Tres son altos: los presupuestos del agente no acotan el gasto en protocolos cuando existe una allowance viva (**BVCC-03**, abierto en V4 y cerrado en V5), cualquiera que llamara primero a la factory abierta podía elegir los guardianes (**BVCC-04**), y una recuperación completada no detenía a los agentes autorizados antes (**BVCC-05**). El resto son medios o menores.

Qué se hizo al respecto

Nueve de los hallazgos de la revisión de V4 se remediaron en V4. Siete exigían cambiar el bytecode y viajaron juntos como la generación **V4**, desplegada y verificada de forma independiente en las seis redes el 2026-07-28. Cuatro quedaron abiertos o aceptados en V4: BVCC-03, el material, y BVCC-09, BVCC-10 y BVCC-13. La V5 cierra los cuatro en el código y conserva todos los arreglos de V4.

Los diecisiete hallazgos de V5 se resolvieron antes del despliegue, cada arreglo con su test de regresión. La capa de validación profunda también está completa en V5: el Universal Router, el PositionManager de Uniswap v4, el Pool de Aave v3, el position manager de Uniswap v3 y el token nativo envuelto tienen un validator activo en todas las redes.

> **Dos advertencias deben acompañar cualquier afirmación de que V5 corrigió esto.** Primera: una smart account desplegada no se puede actualizar por dentro, porque su dirección sale de su código. **Las wallets que siguen en V1 a V4 ejecutan su propio bytecode hasta que sus dueños migren.** BVCC-01 afectaba por igual a V1, V2 y V3, y BVCC-03 sigue abierto en toda wallet V4. La app avisa de una wallet antigua, y una guía de migración más el botón «Enviar todo» mueven los fondos con una firma por red. Segunda: las garantías de V5 descansan en cuatro supuestos de confianza declarados (BVCC-19, BVCC-25, BVCC-29 y BVCC-30). El §6 explica qué pide cada uno al dueño.

> **Esta es una revisión interna.** La hizo el equipo de desarrollo sobre su propio código. Las cinco rondas de V5 y el equipo rojo los ejecutó el equipo con herramientas de análisis con IA; ningún auditor humano independiente ha revisado ninguna generación, y los contratos no han sido auditados externamente. Cuatro de los hallazgos de V4 (BVCC-04, BVCC-06, BVCC-07 y BVCC-11) salieron de una pasada posterior con otra herramienta después de que tres pasadas anteriores no los vieran, y las rondas de V5 siguieron encontrando caminos nuevos hasta la quinta. Ambas cosas dicen lo mismo: **que un hallazgo no aparezca aquí es evidencia débil de que no exista.** El §7 declara los límites de lo que establece este documento.

2 Alcance, metodología y modelo de severidad

Código revisado

ELEMENTO	VALOR
Repositorio	`blockventurechaincapital-crypto/bvcc-wallet-agent`
Commit de **V5**	`838a70fabba4c9ab9ec26a13ea9059a63f75f150`
Árbol de `contracts-v5/src` (**V5**)	`64a5c674e5fdfd1e457daf59c8bf3bb813c89e62`
Commit de **V4**	`a3e28f1a7c8e265d5b3630f98e4c8e77c6752eb5`
Árbol de `contracts/src` (**V4**)	`337e84f05fa11b948e3ad3221398ea4edc030f45`
Build	solc 0.8.36 · `via_ir` · `optimizer_runs = 50` · EVM `cancun` (congelada — las direcciones CREATE2 dependen de ella). V5 además compila sin hash de metadatos, así que la build reproduce byte a byte el bytecode desplegado
Dependencias (V5)	OpenZeppelin Contracts `v5.6.1` (etiqueta oficial) · forge-std `v1.16.1`, solo pruebas
Herramientas	Slither 0.11.3 · Foundry (forge 1.5.1; forge 1.8.6 en las rondas de V5) · viem 2.x

El árbol de V5 contiene el núcleo congelado el 2026-10-29 más dos validators añadidos el 2026-11-02 sin tocar el núcleo (posiciones de Uniswap v3 y token nativo envuelto). Las rondas anteriores revisaron los árboles de V1, V2 y V3 en sus propios commits; esos hallazgos y sus remediaciones están en el Anexo D y el Anexo E.

Dentro del alcance

V5 (`contracts-v5/src`):

- `BVCCSmartWalletV5.sol` — la wallet con passkey: creación con firma de la credencial, ejecución del dueño y comisiones, caducidad de firmas, votos de guardianes y recuperación.
- `BVCCAgentWalletV5.sol` y la librería externa `BVCCAgentLogicV5.sol` — permisos, policies, pausa y caducidad por agente, y el lote medido del agente.
- `BVCCWalletFactoryV5.sol` — un solo contrato de factory, desplegado dos veces, que crea clones ERC-1167 no actualizables por CREATE2.
- `BVCCValidatorRegistryV5.sol`, `BVCCHookRegistryV5.sol`, `IBVCCValidatorV5` y cinco validators: Universal Router, PositionManager de Uniswap v4, Pool de Aave v3, NonfungiblePositionManager de Uniswap v3 y nativo envuelto (WETH, WBNB, WPOL).
- Los scripts de despliegue, porque fijan todas las direcciones y las conexiones entre contratos.

V4 (`contracts/src`):

- `BVCCAgentWallet.sol` — lógica de permisos del agente (`executeAsAgent`, topes por token y de ETH, pausa, caducidad, la capa de call policies).
- `BVCCWallet.sol` — cuenta inteligente base (ejecución, comisiones, recuperación por guardianes).
- `BVCCAgentWalletFactory.sol` / `BVCCWalletFactory.sol` — factories CREATE2.

- `BVCCValidatorRegistry.sol`, `BVCCUniversalRouterValidator.sol`, `BVCCPositionManagerValidator.sol`, `BVCCHookRegistry.sol` y la interfaz `IBVCCValidator`.
- Los presets de call policy que registra la interfaz, en tanto que configuración que determina el comportamiento on-chain.

Fuera del alcance

- Los protocolos de terceros con los que interactúa la wallet (Uniswap, Aave, Permit2): se asume su propia seguridad, y la wallet se revisa partiendo de que pueden comportarse de forma adversarial en su interfaz.
- Infraestructura de bundler, paymaster y EntryPoint.
- La interfaz web, salvo donde registra policy on-chain o es el único sitio donde existe un control (se señala explícitamente cuando es el caso).
- Riesgo económico y de mercado, incluida la manipulación de precios en los pools contra los que opera la wallet, salvo donde es el mecanismo de un hallazgo (BVCC-03).

Metodología

- **Revisión manual** de la vía de ejecución del agente, la contabilidad de permisos, la recuperación y la capa de call policies, en cuatro pasadas a lo largo de las cuatro generaciones.
- **Análisis estático** — Slither 0.11.3 sobre el árbol en cada ronda; resultados y registro de falsos positivos en el Anexo C.
- **Pruebas unitarias, de fork y de fuzz** — Foundry. Cada hallazgo tiene su test de regresión. El validator del router se fuzzea sobre todo su espacio de comandos contra un modelo del router real; Aave se ejercita sobre un fork de mainnet.
- **Desarrollo de exploits** — los hallazgos se confirman con una prueba de concepto que funciona, no con un argumento. BVCC-01 se reprodujo contra una wallet creada desde la factory V2 *viva* sobre un fork.
- **Pruebas on-chain** — la EOA del agente ejecuta `executeAsAgent` directamente. Los reverts se confirman por simulación (`eth_call`) contra estado vivo; los casos válidos, como transacciones reales en Arbitrum Sepolia y Arbitrum One.
- **Verificación del despliegue** — el bytecode desplegado, el owner y el cableado del registry se releen de cada red al margen del script de despliegue.
- **Rondas de revisión de V5** — cinco rondas entre el 2026-10-27 y el 2026-10-29, cada una con una herramienta de análisis con IA contra la especificación escrita y el código. Cada hallazgo se confirmó con un test que fallaba, la mayoría sobre forks de las redes reales, antes de corregirlo; la ronda siguiente empezaba comprobando los arreglos de la anterior.
- **Equipo rojo de V5** — dos sesiones de agentes de IA atacando el código, una desde dentro (como agente, guardián, administrador del registry y dueño) y otra desde fuera sin ningún rol, más 128.000 secuencias aleatorias de acciones comprobadas contra seis invariantes del núcleo. Una revisión aparte, también con IA, cubrió los dos validators añadidos tras el congelado: 14 hipótesis, 13 tests adversariales, sin hallazgos.

Modelo de amenazas

Se declara explícitamente porque cambia cómo deben leerse varios hallazgos. Quien dé por hecho que todo problema exige robar una clave juzgará mal los que no exigen absolutamente nada.

ACTOR	QUÉ SE ASUME
Dueño	De confianza. Tiene la passkey. El compromiso de la passkey queda fuera de alcance: equivale a ser el dueño de la wallet, y para ese caso existe la recuperación por guardianes.

Agente	**Semi-confiable, y a efectos de esta revisión se asume hostil.** Su clave vive en un servidor, en una variable de entorno o un fichero `.env`, corriendo desatendida; y el agente puede ser de un operador tercero que el dueño no controla. «El agente actúa fuera de su mandato» no es una condición previa exótica: es justamente el suceso que el sistema de permisos existe para acotar.
Guardián	Honesto en mayoría. La recuperación exige dos de tres; el diseño acepta que dos guardianes coaligados se queden la wallet si el dueño no cancela en 48 horas.
Tercero	No confiable, no tiene ninguna credencial, y puede llamar a todos los puntos de entrada abiertos —incluidas las factories— en cualquier orden y en cualquier red.
Protocolo	Se asume correcto en sí mismo, pero puede comportarse de forma adversarial en su interfaz: un pool puede estar controlado por el atacante, un token puede reentrar o devolver valores inesperados.

> **Por qué «hace falta la clave del agente» no es un atenuante.** Los topes por token, los presupuestos diario y vitalicio, la caducidad, la pausa y la lista de destinatarios existen precisamente porque el agente no es de fiar del todo. Si una clave de agente comprometida u hostil se tratara como fuera de alcance, toda la capa de permisos no tendría sentido. Los límites que solo se sostienen mientras el agente se porta bien no son límites: son sugerencias. Por eso BVCC-01 es Crítico: convierte «este agente puede gastar 0,003 ETH» en «este agente puede llevárselo todo», que es un fallo de la promesa central del producto y no de una función concreta. Las condiciones previas se recogen hallazgo a hallazgo en el §3 para que el lector pueda ponderar cada una.

Modelo de severidad

La severidad es **impacto × probabilidad**. El impacto es el peor desenlace para un titular si se explota el problema; la probabilidad pesa las condiciones previas que debe cumplir un atacante —compromiso de clave, una carrera, un timelock, una configuración concreta de la wallet— y no lo probable que se considere el ataque en abstracto.

SEVERIDAD	SIGNIFICADO
CRÍTICA	Pérdida de todos los fondos de la wallet, o toma de control completa, con condiciones previas que un atacante puede alcanzar de forma realista.
ALTA	Pérdida de fondos más allá de los límites configurados, o toma de control, pero condicionada a una carrera, un timelock, una acción del dueño o una configuración concreta.
MEDIA	Debilita un control de seguridad, degrada la recuperación o deniega el servicio, sin mover fondos directamente.
BAJA	Viola un invariante declarado con consecuencia práctica limitada.
INFORMATIVA	Sin impacto en el despliegue actual; endurecimiento, o una decisión de diseño que se registra para que no se deshaga en silencio.

La severidad de un hallazgo se indica tal como se encontró, no tal como queda tras la remediación. El estado se sigue por separado, de modo que un Crítico corregido sigue viéndose como un Crítico que se corrigió.

Limitaciones de esta revisión

Se declaran aquí y no al final, porque condicionan cómo debe leerse cada hallazgo. Esta es una revisión **interna**: quienes escribieron el código son quienes lo revisaron. No ha sido auditada externamente. Cuatro de los hallazgos salieron de una pasada posterior sobre el código de V3 con una herramienta de análisis distinta, y cada uno había sobrevivido a tres rondas anteriores: cambiar de herramienta encontró lo que repetir el método no encontraba. Aquella pasada también fue interna

— **ningún tercero independiente ha revisado este código**. Que un hallazgo no aparezca en este documento es, por tanto, evidencia débil de que no exista. El §7 desarrolla qué establece y qué no establece esta revisión.

Las rondas de V5 comparten ese límite y añaden uno propio: se hicieron con herramientas de IA. Eso amplió la cobertura, con diecisiete problemas encontrados antes del despliegue, pero una herramienta revisa aquello a lo que se la apunta, y cuatro de las rondas de V5 encontraron un camino nuevo al lado del arreglo anterior. El equipo rojo y la quinta ronda no encontraron ninguna vía de robo en el núcleo; eso es evidencia sobre los caminos que probaron, no una demostración.

3 Registro de hallazgos

Treinta hallazgos: trece de la revisión de V4 y anteriores (BVCC-01 a BVCC-13) y diecisiete de las rondas de V5 (BVCC-14 a BVCC-30), más dos cerrados en generaciones superadas (Anexo D). La severidad es la que tenía al encontrarse; el estado es a 2026-11-11. Para los hallazgos de la revisión de V4 la columna de estado da las dos generaciones, porque las wallets V4 siguen en uso.

ID	HALLAZGO	REQUIERE	SEVERIDAD	ESTADO
Revisión de V4 y anteriores — 2026-06 → 2026-07				
`BVCC-01`	Reentrada cruzada vía `_currentAgent`	Clave del agente, o un operador de agente hostil	CRÍTICA	Corregido en V4
`BVCC-02`	La máscara de comandos del Universal Router acepta un depósito de puente como swap	Clave del agente, con acceso al router	CRÍTICA	Corregido antes de desplegar
`BVCC-03`	Los presupuestos de token no acotan el gasto en protocolos	Clave del agente + una allowance viva	ALTA	ABIERTO en V4 Cerrado en V5
`BVCC-04`	Apropiación de guardianes a través de la factory abierta	**Nada** — cualquier tercero	ALTA	Corregido en V4
`BVCC-05`	Una recuperación completada no detenía a los agentes existentes	Un compromiso ya en marcha	ALTA	Corregido en V4
`BVCC-06`	`credentialId` sin autenticar en el evento de la factory	**Nada** — cualquier tercero	MEDIA	Corregido en V4
`BVCC-07`	Credencial obsoleta tras una recuperación	**Nada** — sigue a toda recuperación	MEDIA	Corregido en V4
`BVCC-08`	Los guardianes no se podían rotar nunca	**Nada** — operativo	MEDIA	Corregido en V4
`BVCC-09`	Un solo guardián puede bloquear una recuperación indefinidamente	Un guardián malicioso	MEDIA	Aceptado en V4 Cerrado en V5
`BVCC-10`	Pausar o revocar un agente no retira sus allowances	Un spender ya aprobado	MEDIA	ABIERTO en V4 — interfaz Cerrado en V5
`BVCC-11`	`approve` podía llevar ETH saltándose la lista de destinatarios	Clave del agente	BAJA	Corregido en V4
`BVCC-12`	`increaseLiquidity` no comprueba el dueño en el NFPM v3	Clave del agente; inalcanzable tal	INFORMATIVA	Mitigado por exclusión en V4 Cerrado en V5

		como está desplegado		
BVCC-13	Los constructores de los validators aceptan la dirección cero	**Nada** — solo error de despliegue	**INFORMATIVA**	**Aceptado en V4** **Cerrado en V5**
Revisión de V5 — 2026-10-27 → 2026-11-02, todo antes de desplegar o sin despliegue				
BVCC-14	Una lectura de saldo fallida desactivaba el presupuesto de token del agente	Clave del agente + un token permitido cuyo saldo no se puede leer	**ALTA**	**Corregido antes de desplegar**
BVCC-15	Una llamada a protocolo podía mover un token no permitido empaquetado en sus datos	Clave del agente + una aprobación antigua del dueño	**ALTA**	**Corregido antes de desplegar**
BVCC-16	Una llamada a protocolo podía pagar a un destinatario fuera de la lista del agente	Clave del agente + una policy sin ancla de destinatario	**ALTA**	**Corregido antes de desplegar**
BVCC-17	Retirar de Aave y reenviar salía a cero contra el presupuesto	Clave del agente con `withdraw` de Aave	**ALTA**	**Corregido antes de desplegar**
BVCC-18	Pagar la comisión en un token con impuesto al emisor superaba el presupuesto	Clave del agente + un token permitido con impuesto al emisor	**MEDIA**	**Corregido antes de desplegar**
BVCC-19	Un validator mutable podía ampliar permisos sin la espera de 48 horas	Admin del registry + un validator mutable	**MEDIA**	**Aceptado** — regla de despliegue
BVCC-20	No se comprobaba un hook congelado al añadir liquidez	Clave del agente + un hook congelado	**MEDIA**	**Corregido antes de desplegar**
BVCC-21	Liquidez trasladada a una posición ajena por un cargo residual	Clave del agente + una posición ajena que aprueba a la wallet	**ALTA**	**Corregido antes de desplegar**
BVCC-22	Liquidez neteada hacia una posición propia aprobada a un tercero	Clave del agente + una aprobación de NFT del dueño	**ALTA**	**Corregido antes de desplegar**
BVCC-23	Un hook v4 no aprobado movía un token no permitido por el Universal Router	Clave del agente + una aprobación antigua del dueño al hook	**ALTA**	**Corregido antes de desplegar**
BVCC-24	Una retirada del agente ejecutaba un hook congelado	Clave del agente + hook congelado + aprobación antigua del dueño	**ALTA**	**Corregido antes de desplegar**
BVCC-25	Un hook v4 aprobado ejecuta código que la	Un hook aprobado	**ALTA**	**Aceptado** — decisión de confianza

	wallet no puede acotar	malicioso o comprometido		
BVCC-26	Un agente podía vaciar la posición de un tercero que aprobó a la wallet	Clave del agente + una aprobación de NFT de un tercero	ALTA	Corregido antes de desplegar
BVCC-27	Los agentes no podían quemar nunca una posición v4	**Nada** — uso normal	MEDIA	Corregido antes de desplegar
BVCC-28	`pauseAgents` revertía si la wallet ya estaba en pausa	**Nada** — operación del dueño	BAJA	Corregido antes de desplegar
BVCC-29	Un token permitido que miente sobre su saldo o mueve otro token	Un token permitido malicioso	MEDIA	**Aceptado** — supuesto de confianza
BVCC-30	La wallet solo mide los tokens que ve o que declara un validator	Un validator que declara de menos	BAJA	**Aceptado** — regla de validators

Por severidad y por estado

SEVERIDAD	TOTAL	REVISIÓN V4	REVISIÓN V5
CRÍTICA	2	2	0
ALTA	13	3	10
MEDIA	10	5	5
BAJA	3	1	2
INFORMATIVA	2	2	0

ESTADO	EN V5	EN WALLETS V4
Corregido o cerrado	26	9 de 13
Abierto	0	2
Aceptado y documentado	4	2

De dónde salen los identificadores de V5

Cada ronda de V5 usó su propia numeración. Aquí se renumeran para continuar la serie; se conservan los identificadores originales para poder seguir los informes de cada ronda.

RONDA	FECHA	IDS ORIGINALES	EN ESTE INFORME	RESULTADO
V5, ronda 1	2026-10-27	`V5-H-01…04` · `V5-M-01…03`	BVCC-14 … BVCC-20	4 altos, 3 medios
V5, ronda 2	2026-10-28	`R2-H01`	BVCC-21	Arreglos de la ronda 1 comprobados; 1 alto

V5, ronda 3	2026-10-28	`R3-H01 · R3-H02`	BVCC-22, BVCC-23	2 altos
V5, ronda 4	2026-10-29	`R4-H01 · R4-H02`	BVCC-24, BVCC-25	2 altos (uno aceptado)
Equipo rojo	2026-10-29	`RT-01 · RT-55 · RT-30-A · RT-52/54 · RT-65`	BVCC-26 … BVCC-30	Ninguna vía de robo en el núcleo; 3 corregidos, 2 aceptados
V5, ronda 5	2026-10-29	—	—	Arreglos del equipo rojo comprobados; sin hallazgos nuevos; código congelado
Validators nuevos	2026-11-02	—	—	Uniswap v3 y nativo envuelto: 14 hipótesis, 13 tests, sin hallazgos

Cómo leer el registro. La severidad se anota como se encontró, así que un crítico ya corregido se sigue leyendo como crítico. La pregunta del lector suele ser «de qué era capaz este código», no solo «qué queda». En V5 no hay nada abierto: veintiséis hallazgos están cerrados en el código y cuatro son supuestos de confianza aceptados que el §6 detalla. En una wallet V4, **BVCC-03** sigue abierto y es el motivo para migrar. La columna **Requiere** importa tanto como la severidad. La mayoría de los hallazgos de V5 necesitan un agente autorizado más algo que el dueño hizo antes, normalmente una aprobación antigua de un token o de un NFT, que el modelo de amenazas del §2 trata como comportamiento normal. Uno, BVCC-26, ponía en riesgo la posición de un tercero y no la del dueño.

4 Hallazgos en detalle

BVCC-01 a BVCC-13 salen de la revisión de V4 y anteriores; donde la V5 cambió el resultado, una nota «Cerrado en V5» explica cómo. BVCC-14 a BVCC-30 salen de las rondas de V5 y van después.

Cada hallazgo se expone con la severidad que tenía al encontrarlo, las condiciones previas que necesita un atacante y la evidencia que lo estableció. La remediación y su verificación van en §5.

El código de V3 se sometió a otra pasada interna con una herramienta de análisis distinta, que planteó cuatro cuestiones. Las cuatro se reprodujeron contra el código en vez de darlas por buenas a partir de su descripción; los tests están en `test/ThirdPartyClaims.t.sol`. Las cuatro se sostienen, aunque dos son más estrechas de lo que se enunciaba y una es una propiedad de ERC-20 más que de esta wallet. Ninguna había aparecido en las tres rondas internas.

BVCC-01 **CRÍTICA** Corregido en V4

Reentrada cruzada vía `_currentAgent`

UBICACIÓN	`BVCCAgentWallet.sol` — `executeAsAgent` / `execute`
IMPACTO	Pérdida total de los fondos. Se saltan todos los límites configurados en una sola transacción; una prueba de concepto movió **49,925 ETH** contra un presupuesto vitalicio de 0,003 ETH.
PROBABILIDAD	Media. La dirección del agente tiene que adquirir código después de ser autorizada: una delegación EIP-7702 que el propio agente firma con su clave, o un despliegue CREATE2 a una dirección pre-comprometida. No hace falta ninguna acción del dueño ni hay aviso.
AFECTA A	V1, V2 y V3 por igual — confirmado drenando una wallet creada desde la factory V2 *viva* sobre un fork de Arbitrum One. Las wallets que no hayan migrado a V4 siguen expuestas.

Este resultado se descartó en un primer momento alegando que un agente es un EOA y que no se le puede ceder el control a mitad de transacción. Ese razonamiento no se sostiene, y el hallazgo queda confirmado.

Mecanismo

`executeAsAgent` fija `_currentAgent = agent` alrededor de su llamada a `super.execute()`, y `_erc7821AuthorizedExecutor` concede la vía `execute()` de nivel propietario a cualquier llamante que cumpla `caller == _currentAgent`. El `execute()` padre no lleva guard de reentrada — `nonReentrant` está solo en `executeAsAgent`— y no aplica ningún límite de agente, porque las comprobaciones viven en `executeAsAgent`. La premisa del EOA descansa en `AgentMustBeEOA` (`agent.code.length == 0`), que se exigía **solo en el momento de autorizar** y nunca se volvía a comprobar al ejecutar. Una dirección puede adquirir código después: mediante una delegación EIP-7702, firmada con la propia clave del agente —justo la clave que el modelo de amenaza asume que un atacante puede tener— o desplegando en una dirección CREATE2 precalculada que estaba sin código cuando se autorizó.

> **Alcance: afecta igual a V1, V2 y V3.** No es una regresión de V3. Los cuatro ingredientes —la bandera, el override del executor, el control de EOA solo al autorizar y el `execute()` padre sin guard— están literalmente en las tres generaciones; lo único que se movió son los números de línea. Se confirmó contra código desplegado, no contra el fuente: sobre un

> fork de Arbitrum One, una wallet creada desde la **factory V2 viva** (`0x8D9e2402…`) fue drenada con el mismo ataque (`test/LegacyV2ReentrancyFork.t.sol`).

Reproducción

`test/AgentReentrancyPoC.t.sol`. Se autoriza un agente sin código con topes de 0,001 / 0,002 / 0,003 ETH (por tx / diario / vitalicio). Después se coloca código en la dirección del agente, modelando la delegación. El agente manda un batch cuyo único elemento es una transferencia de **1 wei** a sí mismo —dentro de todos los topes— que le cede el control; desde ahí llama a `execute()` de vuelta y saca **49,925 ETH** de un saldo de 100 ETH. La diferencia de 0,075 ETH es la comisión propia del 0,15% de la wallet, lo que confirma que el drenaje viajó por la vía de propietario.

Radio de impacto

Se saltan de golpe todas las restricciones del agente: por transacción, diaria, de período y vitalicia, la lista blanca de destinatarios y las call policies de V3. Convierte un agente acotado en control equivalente al del propietario, un agujero estrictamente mayor que el vector de exfiltración que V3 vino a cerrar. No afecta a la vía biométrica del propietario, ni permite nada a un tercero que no tenga la clave del agente.

Remediación — dos capas

Se implementaron tres candidatos y se midieron contra tres formas de ataque, en vez de discutirlos. S1 manda 1 wei directamente al agente; S2 deja al agente sin código al entrar y hace que el propio batch le despliegue código a esa dirección mediante un ayudante whitelisteado (sin EIP-7702 de por medio); S3 no nombra al agente en ningún momento y le hace llegar valor a través de un intermediario permitido.

CANDIDATO	S1 DIRECTO	S2 CREATE2	S3 INDIRECTO	TAMAÑO FACTORY
Ninguno (antes del fix)	drena	drena	drena	`24.283`
B — re-chequear `code.length`	bloquea	**drena**	bloquea	`24.294`
C — el batch no puede apuntar al agente	bloquea	bloquea	**drena**	`24.312`
A — guard en el `execute()` externo	bloquea	bloquea	bloquea	`24.317`
A + B (lo que se publica)	bloquea	bloquea	bloquea	`24.328`

Los dos candidatos descartados fallan por motivos instructivos. **B por sí sola** es otra comprobación puntual —la misma clase de suposición que causó el bug— y S2 se le cuela haciendo que el código aparezca *durante* el batch. **C por sí sola** cierra únicamente la puerta que usó la primera prueba de concepto; S3 llega al agente a través de un protocolo legítimo.

Lo que se publica son las dos capas. La **capa 1** vuelve a comprobar el invariante del EOA en cada ejecución, de modo que se exige de forma continua y no una sola vez. La **capa 2** sobreescribe `execute()` en la wallet de agente para rechazar cualquier entrada externa mientras hay un batch de agente en vuelo (`ReentrantAgentExecute`); `executeAsAgent` llega al padre por `super.execute()`, que resuelve estáticamente y por tanto salta el override, así que el camino legítimo queda intacto. La capa 2 es la que no depende de *cómo* ni de *cuándo* consiguió código el agente: cierra la ventana en sí. Cada capa tiene su propio test de regresión,

y la capa 2 queda probada de forma independiente con la forma S2, donde la capa 1 pasa limpiamente y el drenaje se detiene igual.

Coste: la factory de agentes crece 45 bytes hasta 24.328, dejando 248 de margen bajo EIP-170, y `executeAsAgent` en caso 3 sube de 64.758 a 64.903 gas —145 gas, un 0,2%, porque la dirección del agente ya está caliente al ser quien firma la transacción—. Una consecuencia deliberada de comportamiento: un agente que adopte una delegación EIP-7702 por motivos ajenos dejará de funcionar hasta que la quite. Es el invariante que el propio contrato declara, ahora exigido.

> **Desplegado en V4 — pero solo para las wallets creadas en V4.** El arreglo cambia el bytecode de la wallet y por tanto las direcciones CREATE2, así que no puede alcanzar a una wallet ya desplegada; que `Create2Consistency.t.sol` fallara contra las constantes viejas fue la prueba mecánica de eso. Las factories V4 están vivas en las seis redes, **y toda wallet que siga en V1, V2 o V3 ejecuta bytecode vulnerable hasta que su dueño migre. Mitigación para esas wallets, sin redesplegar:** autorizar a los agentes con una `allowedRecipients` no vacía que no contenga la dirección del propio agente, lo que le quita al batch la capacidad de devolverle el control (`test_RecipientWhitelist_BlocksTheEntry`). Las wallets que se queden en el default permisivo —lista de destinatarios vacía, es decir, cualquier destino— son la configuración expuesta. Pausar o revocar al agente también lo cierra de inmediato.

Entre los Low, 20 `calls-loop` y 4 `timestamp` son inherentes a la ejecución por batches y a los límites indexados por día; el `return-bomb` de `_tryBalanceOf` está acotado por el mismo `PROBE_GAS_CAP` que introdujo V2, que también limita cuántos datos de retorno puede permitirse producir un token hostil. Un resultado sí es una carencia real, aunque inerte:

> **Detalle aceptado — sin zero-check en los constructores de los validators.** Ni `BVCCUniversalRouterValidator` ni `BVCCPositionManagerValidator` rechazan una dirección cero para el protocolo al que se atan, a diferencia de los constructores de las factories endurecidos en V1 (`ZeroOwner`). Un validator atado a la dirección cero denegaría todas las llamadas —fail-closed, así que el comportamiento desplegado es seguro— y las seis instancias desplegadas llevan los argumentos de constructor correctos, registrados con sus codehashes en `contracts/deployments/` y verificados en los exploradores. Añadir el control cambiaría el bytecode, y por tanto las direcciones CREATE2, forzando un redespliegue y un ciclo de gobernanza de 48 horas por red. Queda anotado aquí para la próxima revisión de bytecode, no ejecutado ahora.

> **Re-ejecución tras los arreglos de V4: 53 resultados — 3 High, 27 Low, 23 Informational.** Los tres High no cambian, y los tres son ya falsos positivos: los dos `uninitialized-state` son mappings, y `reentrancy-eth` sigue viendo la bandera escrita tras la llamada externa aunque el BVCC-01 cerrara la ventana — ahora incluso lista `execute()` entre las funciones alcanzables, que es precisamente donde está el guard. Apareció un segundo `dead-code` por `_afterRecovery()`, el hook de recuperación del BVCC-05: el mismo falso positivo que `_feeNumerator()`, una función virtual vacía que sobreescribe el hijo. Ninguna de las dos se toca. Los arreglos no introdujeron nada nuevo. **El análisis estático detecta patrones, no emite un veredicto**: esta cifra no se movió mientras la reentrada fue explotable durante tres generaciones, y tampoco se movió cuando el arreglo la cerró.

Los resultados Informational son de las mismas clases que en las pasadas de V1 y V2: ensamblador inline, low-level calls, convenciones de nombres en los inmutables, dispersión de pragmas por el árbol de OpenZeppelin, y el aviso de "dead code"

sobre `_feeNumerator()` — falso positivo conocido: se alcanza por dispatch virtual y fija la comisión de agente del 0,15%.

BVCC-02 **CRÍTICA** Corregido antes de desplegar

La máscara de comandos del Universal Router acepta un depósito de puente como swap

UBICACIÓN	`BVCCUniversalRouterValidator.sol` — decodificación de comandos
IMPACTO	Crítico. El router habría enviado los fondos de la wallet a un atacante en otra cadena mientras el validator clasificaba la llamada como un swap normal: justo la clase de exfiltración que el validator existe para impedir.
PROBABILIDAD	Media — una clave de agente comprometida con acceso al Universal Router, y nada más.
NOTA	Encontrado en revisión **antes** de registrar el validator en ninguna red, así que nunca fue explotable en producción. Se registra porque lo interesante es el casi-fallo: el validator se escribió para impedir justo esto y lo habría permitido.

> **Bug crítico detectado en revisión — la máscara de comandos.** La primera versión del validator enmascaraba los bytes de comando con `& 0x3f`. El router real enmascara con `& 0x7f` (el bit `0x80` es `FLAG_ALLOW_REVERT`), lo que significa que `0x40` **no** es un alias de `0x00`: es `ACROSS_V4_DEPOSIT_V3`, un depósito en un puente cross-chain. Con la máscara equivocada el validator lo habría aceptado como un swap v3 mientras el router bridgeaba los fondos de la wallet hacia un atacante en otra cadena: exactamente la clase de exfiltración que V3 existe para impedir. La corrección sustituyó la máscara por coincidencia exacta, ató el validator a un único router y rechazó las cadenas de comandos vacías. El comportamiento se fuzzea ahora sobre los 256 bytes de comando contra un modelo del router, y se volvió a confirmar en mainnet tras el despliegue (ver Anexo B).

BVCC-03 **ALTA** **ABIERTO**

Los presupuestos de token no acotan el gasto en protocolos

UBICACIÓN	`BVCCAgentWallet.sol` — `_validateExecutionItem`, caso 3
IMPACTO	Crítico donde aplica: pérdida de la allowance viva entera, con independencia del presupuesto de token configurado para el agente.
PROBABILIDAD	Media-baja. Exige a la vez una clave de agente comprometida *y* una allowance que el dueño ya haya concedido a un protocolo alcanzable por el agente. Una wallet sin allowance viva no está expuesta.
NOTA	El único hallazgo de la revisión de V4 que V4 no cierra. **Cerrado en V5** midiendo lo que sale; ver la nota al final de este hallazgo. Las wallets V4 siguen expuestas.

Los contadores por token solo acumulan para los selectores `transfer` y `approve` de ERC-20: a `_checkTokenWhitelistAndAmount` se llega desde esas dos ramas y desde ninguna más. Una llamada de caso 3 gasta por tanto tokens sin tocar `tokenMaxAmounts`, el límite diario ni el presupuesto vitalicio — y sin comparar el activo que viaja dentro de su calldata con `allowedTokens`. Esto es deliberado hasta cierto punto: el modelo cobra el presupuesto en el paso del `approve`, que es lo que afirma la batería de Aave sobre fork, así que en el flujo habitual de aprobar y luego operar el presupuesto sí se consume.

El hueco son las **allowances previas**, y el caso común no es rebuscado: un propietario que haya concedido una allowance amplia a un router o a Permit2 desde la interfaz —práctica

estándar— deja al agente moviendo ese token por la vía de caso 3 indefinidamente sin consumir una unidad de su presupuesto. Reproducido en `test_Claim2_PreExistingAllowanceEscapesTokenBudgets`: con los tres topes de token puestos a 10, el agente mueve 500 a través de un router whitelisteado y ambos contadores se quedan a cero. El propio `approve(address,address,uint160,uint48)` de Permit2 es asimismo un selector distinto y tampoco cuenta.

> **Corrección de severidad.** Un borrador anterior de esta sección describía la pérdida como operar por encima del volumen previsto, apoyándose en que las call policies anclan todos los destinatarios a la wallet. Eso se queda corto. El anclaje garantiza *dónde aterriza el output*, no *cuánto vale*: un agente comprometido puede cambiar un activo valioso por uno sin valor a través de un pool que él controla, y el token inútil vuelve obedientemente a casa mientras el valor se queda en la liquidez del atacante. Nada en el diseño actual se opone — el validator del Universal Router no contiene ni una sola referencia a cantidades o precios, y las policies de SwapRouter02 anclan el destinatario y nada más, así que un swap con `amountOutMinimum = 1` pasa todos los controles. Donde exista una allowance viva, esto es un vaciado completo de los tokens que cubra, no deslizamiento de precio.

La selección de tokens lo estrecha, pero no lo cierra. Elegir qué tokens puede tocar un agente sí gobierna `transfer` y `approve`, de modo que un token no seleccionado y sin allowance viva no se puede mover: el agente tampoco puede aprobarlo. Quedan dos huecos. Un token que ya arrastra una allowance es alcanzable al margen de la selección. Y el `borrow` y el `withdraw` de Aave no consumen allowance alguna, así que ahí no hay nada que poner a cero: un agente con policy de Aave puede endeudar la wallet en un activo que el propietario nunca seleccionó, contra su propio colateral, con el riesgo de liquidación que eso implica. Los fondos aterrizan en la wallet —es daño, no robo— pero es justo el daño que la selección de tokens debería haber evitado.

Plan de remediación

La contabilidad completa dentro de la wallet exigiría que los validators profundos decodifiquen y devuelvan los pares `(token, gasto máximo)` que ven, acumulándolos la wallet por `_applyTokenSpend` y contando `amountInMaximum` en las rutas de salida exacta. Eso toca todos los validators más la wallet, y la factory de agentes tiene 101 bytes de margen bajo EIP-170. Por eso se escalona en vez de intentarse ahora.

FASE	MEDIDA	COSTE
Ahora solo interfaz	Renombrar el límite para que deje de sugerir un tope global — gobierna transferencias y autorizaciones directas, no el volumen DeFi	Ninguno
	Al autorizar, leer las allowances de cada token relevante hacia cada protocolo que las policies del agente puedan alcanzar, y exigir una revocación firmada de las que no sean cero	Frontend + un batch firmado
	Ejecutar las operaciones del agente como batches atómicos: `approve(exacto)` → llamada al protocolo → `approve(0)`, de modo que la primera pata consuma presupuesto y nada sobreviva al batch	Frontend; funciona con el contrato actual
	Para Permit2, comprobar las dos capas —la allowance ERC-20 hacia Permit2 y la allowance interna de Permit2 hacia el router— con cantidades exactas y expiraciones cortas	Frontend

	Una pantalla de emergencia para el dueño que revoque todas las allowances, y saldos moderados en las wallets que lleven agentes	Frontend + operativa
Después capa de validators	Control de precio en los validators de swap: rechazar una ruta cuyo `amountOutMinimum` sea inverosímil frente a un oráculo o un TWAP. Un validator es una función `view`, así que puede consultarlo. **Esta es la medida que cierra el vaciado por pool amañado.**	Validators nuevos; 48 h de gobernanza por red
	Un validator de Aave que compruebe el argumento `asset` contra los tokens seleccionados de la wallet, para que la selección gobierne también el préstamo	Validator nuevo; mismo ciclo
Más adelante	Un vault de agente por el que el agente esté obligado a pasar, de modo que la cantidad entregada sea un techo físico y los routers salgan de `allowedProtocols`	Contrato nuevo + integración
	Contabilidad dentro de la wallet, cuando un rediseño de la factory libere el bytecode necesario	Trabajo de escala V5

Cómo baja la severidad. La calificación viene de dos cosas: la pérdida no tiene techo, y la condición previa es comportamiento normal del usuario. Fíjese en lo que ocurre cuando no existe ninguna allowance viva — el agente tiene que hacer `approve` antes de poder gastar, y ese approve *sí* se cuenta contra el presupuesto por token y pasa por las listas de tokens y destinatarios. El presupuesto vuelve a acotarlo todo. El hallazgo no es por tanto «el presupuesto no funciona» sino «el presupuesto no funciona *mientras exista una allowance viva*», y esa condición es atacable sin tocar los contratos.

ESTADO	QUÉ QUEDA ACOTADO	SEVERIDAD
Hoy	Nada, si existe una allowance	**ALTA**
+ higiene de allowances (a cero al autorizar, approvals exactos y atómicos, expiraciones cortas en Permit2)	El gasto acumulado, vía el presupuesto de token	MEDIA
+ control de precio en los validators de swap	También la pérdida por operación	BAJA

La higiene sola se queda en media porque es una foto: el dueño puede conceder una allowance desde cualquier dApp después. Baja la probabilidad; solo el validator baja el impacto. Dos salvedades: el `borrow` y el `withdraw` de Aave no consumen allowance, así que la higiene no llega ahí hasta que un validator de Aave compruebe el `asset`; y un límite de precio acota cada operación, no el total — acotar el gasto acumulado de caso 3 exige la contabilidad en la wallet que el bytecode no puede albergar hoy.

Lo que las medidas de interfaz no pueden hacer. Impedir approvals ilimitados mientras haya un agente activo no es aplicable desde el frontend de la propia wallet: el propietario puede conceder una allowance desde cualquier dApp, y nada on-chain lo impide. La comprobación en el momento de autorizar es por tanto una foto que cierra el caso común —la allowance olvidada— no un invariante. Solo la capa de validators lo convierte en uno.

Cerrado en V5. La V5 tomó la fila «contabilidad dentro de la wallet» del plan anterior y la metió en el núcleo, midiendo en lugar de decodificar. Cada lote de un agente lee el saldo de todo token permitido, todo token encontrado en los datos y todo token que declare un validator, antes y después de ejecutarse. Lo que haya salido de la wallet se carga contra

los límites por lote, diario y total del agente, con la comisión del 0,15 % incluida. Una allowance viva ya no importa: un swap que usa una aprobación que el dueño dio antes se cobra como cualquier otro gasto, como se probó sobre un fork de Arbitrum con los contratos reales de Uniswap. Un swap hacia un token sin valor se cobra igualmente por todo el valor que salió, así que la pérdida queda acotada por el presupuesto del agente. La V5 no añade un control de precio. Aave también queda cubierto: `borrow` se carga al presupuesto, el activo debe estar en la lista del agente, y pedir prestado, retirar o cambiar el colateral se rechaza si el factor de salud quedara por debajo de 1,2. Las retiradas se cobran al entrar (BVCC-17). El mecanismo nuevo trajo sus propios hallazgos, de BVCC-14 a BVCC-30, todos resueltos antes del despliegue.

BVCC-04 ALTA Corregido en V4

Apropiación de guardianes a través de la factory abierta

UBICACIÓN	`BVCCWalletFactory.sol` / `BVCCAgentWalletFactory.sol` — `createWallet`; `BVCCWallet.sol` — `setGuardians`
IMPACTO	Crítico. Toma de control completa: los guardianes del okupa rotan al dueño en cuanto pasa el timelock de recuperación.
PROBABILIDAD	Baja. El atacante tiene que desplegar la dirección de la víctima primero en una red que esta no haya usado, la víctima tiene que fondearla, y deben pasar las 48 horas sin que el dueño cancele. Detectable leyendo `guardians()` antes de meter fondos.
REPORTADO POR	Pasada interna posterior, herramienta distinta

La dirección CREATE2 de una wallet se deriva únicamente de la clave pública de la passkey; los guardianes no forman parte de ella. `createWallet` no exige autorización alguna y fija los guardianes en la misma llamada, y `setGuardians` es de un solo uso. Quien despliegue una dirección primero elige, de forma permanente, a las tres partes que pueden rotar a su propietario. Si la wallet ya existe, la factory la devuelve sin cambios, así que quien llegue segundo recibe la wallet ocupada sin error y sin ninguna señal.

La apropiación completa se reprodujo de principio a fin (`test_Claim1_GuardianSquattingTakesOverTheWallet`): un atacante despliega la dirección determinista de la víctima con sus propios guardianes; la app de la víctima «crea» después la wallet y recibe de vuelta esa misma dirección; dos de los guardianes del atacante abren una recuperación, agotan el timelock de 48 horas y rotan el signer. La wallet y su saldo son suyos.

El diseño multichain amplifica esto precisamente por ser determinista: la clave pública de la passkey queda visible on-chain en cuanto el usuario despliega en cualquier red, y la misma dirección es reclamable en todas las demás. La exposición son, por tanto, **las redes donde un usuario aún no ha desplegado**, no las wallets ya en uso — una vez fijados los guardianes correctos no se pueden sustituir. El timelock de 48 horas junto con `cancelRecovery` es una defensa real, pero solo en una red que el propietario esté vigilando, lo que por definición excluye aquellas a las que no ha desplegado.

> **Una parte de la recomendación del revisor no se sostiene.** Rechazar guardianes duplicados on-chain es higiene, no un agujero: las aprobaciones se llevan por dirección, así que un guardián duplicado no puede aprobar dos veces ni alcanzar por sí solo el umbral de 2 de 3 (`test_Claim1_DuplicateGuardianCannotSelfApprove`). Los duplicados degradan el esquema a 2 de 2; no permiten una apropiación en solitario.

Corrección, ya aplicada en código. La factory ya no elige guardianes: despliega la wallet sin ninguno, y `setGuardians` queda protegida con `msg.sender == address(this)` —el mismo patrón de auto-llamada que ya usan `authorizeAgent` y `setCallPolicy`— de modo que solo una operación firmada con la passkey puede fijarlos. Los duplicados también se rechazan ahora. El ocupante se queda con un cascarón vacío que no puede configurar y, de hecho, ha pagado el gas del despliegue de la víctima. Los guardianes no se leen en ningún sitio salvo en la ruta de recuperación, así que una wallet sin ellos opera con normalidad; lo único no disponible es la recuperación hasta que el dueño firme. Lo cubren siete tests de regresión, incluidos la vía real del propietario —un batch de `execute()` cuyo item apunta a la propia wallet— y el hecho de que un agente autorizado tampoco llega, porque `executeAsAgent` rechaza cualquier item dirigido a la wallet.

BVCC-05 ALTA Corregido en V4

Una recuperación completada no detenía a los agentes existentes

UBICACIÓN	`BVCCWallet.sol` — `executeRecovery`; `BVCCAgentWallet.sol` — `_afterRecovery`
IMPACTO	Alto. El agente del atacante sigue gastando después de que el dueño crea haberlo dejado fuera — y el compromiso es justo el motivo por el que un dueño recurre a la recuperación.
PROBABILIDAD	Media — aplica a cualquier recuperación hecha por un compromiso, que es la razón principal de que el mecanismo exista.

La recuperación no revocaba los agentes. CORREGIDO EN V4 `executeRecovery` rotaba el signer y no tocaba nada más, así que un agente autorizado antes de la recuperación seguía gastando después, con su presupuesto intacto. Importa porque el compromiso es justamente el motivo por el que un dueño recurre a la recuperación: quien recuperaba creyendo haber echado al atacante no le había revocado el agente. Ahora la recuperación pausa los agentes mediante un hook `_afterRecovery`. Pausar en vez de revocar es deliberado: gas constante sea cual sea la lista de agentes, reversible, y conserva la configuración de cada uno para que el nuevo dueño la juzgue en vez de tener que reconstruirla. La guarda es `if (!paused()) _pause()`: sin ella, una wallet con los agentes ya pausados no habría podido completar una recuperación.

BVCC-06 MEDIA Corregido en V4

`credentialId` sin autenticar en el evento de la factory

UBICACIÓN	Evento `WalletCreated` de la factory; `BVCCWallet.sol` — `_setCredential` / `setCredentialId`
IMPACTO	Medio. Un okupa podía publicar una credencial falsa para la dirección de otro, y tras una recuperación la credencial on-chain seguía apuntando a la passkey recién sustituida: la interfaz ofrecería la clave equivocada.
PROBABILIDAD	Media — el primero se deriva de BVCC-04; el segundo, de cualquier recuperación completada.
REPORTADO POR	Pasada interna posterior, herramienta distinta — revisión de seguimiento

Al corregirlo aparecieron dos hallazgos emparentados. El `credentialId` viajaba por la factory como campo de evento sin autenticar, así que un ocupante podía publicar uno falso para la dirección de otro. No puede costar fondos —las firmas se validan contra la clave pública guardada— pero un cliente que se fíe preselecciona una credencial que el dueño no tiene, y el

prompt de la passkey falla. Se ha sacado de la factory por completo: ahora lo anuncia la propia wallet, en la misma llamada firmada que fija los guardianes, mediante `CredentialSet(bytes32 indexed credentialHash, bytes credentialId)`. El hash va indexado para que un cliente con el `rawId` de una passkey pueda encontrar la wallet a la que pertenece.

BVCC-07 MEDIA Corregido en V4

Credencial obsoleta tras una recuperación

UBICACIÓN	`BVCCWallet.sol` — `executeRecovery` / `setCredentialId`
IMPACTO	Medio. Tras una recuperación por guardianes la credencial on-chain seguía apuntando a la passkey que el dueño ya no tenía, así que un cliente que se fíe pide la clave equivocada y la wallet recuperada parece inservible.
PROBABILIDAD	Segura tras cualquier recuperación completada: no es un ataque, es un estado final roto del mecanismo que existe para rescatar la wallet.
REPORTADO POR	Pasada interna posterior, herramienta distinta — revisión de seguimiento

Detectado al corregir BVCC-04. `executeRecovery` rota el signer pero dejaba la credencial intacta, de modo que tras una recuperación por guardianes la credencial on-chain apuntaba a una passkey que el dueño ya no tenía; `setCredentialId`, también con auto-llamada, lo cierra.

BVCC-08 MEDIA Corregido en V4

Los guardianes no se podían rotar nunca

UBICACIÓN	`BVCCWallet.sol` — `setGuardians`
IMPACTO	Medio. Un guardián cuya clave se perdiera o se comprometiera después era permanente durante toda la vida de la wallet; el único remedio era crear otra wallet y mover los fondos.
PROBABILIDAD	Baja como ataque, segura como fallo operativo en un horizonte suficientemente largo.

Los guardianes no se podían rotar nunca. CORREGIDO EN V4 `setGuardians` era de un solo uso, así que un guardián cuya clave se perdiera o se comprometiera después era permanente durante toda la vida de la wallet, y el único remedio era crear otra wallet y mover los fondos. El conjunto es ahora reemplazable por el dueño. Sigue siendo una auto-llamada, de modo que la apropiación del BVCC-04 continúa cerrada, y la rotación se rechaza con una recuperación en vuelo (`RecoveryActive`) — si no, una passkey robada podría cambiar los guardianes por debajo de una recuperación ya en marcha. El dueño cancela primero y rota después.

BVCC-09 MEDIA Aceptado

Un solo guardián puede bloquear una recuperación indefinidamente

UBICACIÓN	`BVCCWallet.sol` — `initiateRecovery`
IMPACTO	Medio. Denegación de servicio contra el mecanismo de rescate. No se mueve ningún fondo y el dueño no se rota nunca.
PROBABILIDAD	Media — exige un guardián malicioso o comprometido dispuesto a hacer front-run a cada segunda aprobación.
NOTA	Aceptado como limitación documentada: es una carrera, no un bloqueo, y en cuanto el par honesto consigue dos aprobaciones nadie puede reiniciarlo.

Un solo guardián puede bloquear una recuperación indefinidamente.

ABIERTO — ACEPTADO `initiateRecovery` se puede llamar siempre que haya menos de dos aprobaciones, y sobrescribe la clave pendiente y pone el timelock a cero. Un guardián malicioso solo tiene por tanto que adelantarse a cada segunda aprobación para que la pareja honesta no alcance nunca el umbral. Es denegación de servicio contra el mecanismo de rescate, no robo: el dueño no llega a rotar. Y es una carrera, no un candado — un test acompañante demuestra que en cuanto la pareja honesta mete dos aprobaciones, ya nadie puede reiniciarla. Aceptado como limitación documentada.

> **El modelo de confianza, sin adornos.** Dos guardianes coludidos se quedan con la wallet si el dueño no cancela dentro de la ventana de 48 horas — eso es lo que hace que la recuperación funcione, y así está probado. Implica que el dueño debe vigilar todas las redes donde exista su wallet, que es el mismo requisito operativo que la apropiación de guardianes del BVCC-04 dejó al descubierto por otra vía.

> **Cerrado en V5.** La recuperación ya no tiene un paso de «iniciar» que cualquiera pueda sobrescribir. Cada guardián vota una propuesta concreta (clave y credencial nuevas) y solo controla su propio voto. Dos votos iguales arman la propuesta y empiezan las 48 horas; un tercer guardián no puede reiniciar ni borrar los votos de los otros, y la propuesta sigue armada mientras tenga dos votos. Un guardián hostil puede como mucho retrasar un ciclo votando y retirando su voto. 128.000 acciones aleatorias de guardianes confirmaron los invariantes: armada exactamente cuando coinciden dos votos, ningún guardián cambia una propuesta que no sostiene, y la clave nunca cambia antes de 48 horas. El dueño puede cancelar siempre con su passkey.

BVCC-10 MEDIA ABIERTO — interfaz

Pausar o revocar un agente no retira sus allowances

UBICACIÓN	Semántica de allowance de ERC-20; flujo de revocación en la interfaz
IMPACTO	Medio. Un dueño que pausa un agente comprometido no ha detenido al spender que ya aprobó. La allowance sobrevive en el contrato del token.
PROBABILIDAD	Media — toda revocación hecha bajo la suposición razonable de que basta con eso.
NOTA	Inherente a ERC-20: nada que haga la wallet puede retirar una allowance que guarda el token. El remedio es una interfaz que revoque explícitamente — ver §6.

Una allowance vive en el contrato del token, así que nada de lo que haga la wallet puede retirarla. Reproducido en `test_Claim3_AllowanceSurvivesPauseAndRevoke`: el agente aprueba dentro de su tope, el propietario pausa *y* revoca, y el spender se lleva igualmente el importe completo. El revisor señala correctamente que el importe se cargó al presupuesto en el momento de aprobar, así que no se elude ningún límite. Es una propiedad de cualquier smart account, no un defecto de esta, y afecta a la respuesta ante incidentes más que al modelo de permisos: tras revocar un agente, el propietario debería además poner a cero las allowances que le concedió. Un aviso explícito al pausar y una pantalla de revocación de allowances lo cubren.

> **Cerrado en V5.** Un agente ya no puede dejar una allowance viva. Al final de cada lote del agente, cada aprobación que hizo vuelve a su valor anterior (pasando antes por cero, para tokens como USDT), así que las aprobaciones del propio dueño no se tocan. Un `approve` del agente solo puede nombrar un protocolo de su lista, nunca un destinatario cualquiera.

> Las allowances que da el *dueño* siguen viviendo en los contratos de los tokens; las gestiona él, y el agente no puede usarlas por encima de su presupuesto.

BVCC-11 **BAJA** **Corregido en V4**

`approve` podía llevar ETH saltándose la lista de destinatarios

UBICACIÓN	`BVCCAgentWallet.sol` — `_validateExecutionItem`
IMPACTO	Bajo. Viola el invariante que el propio contrato declara —las llamadas de token no llevan valor—; la consecuencia práctica está acotada por el presupuesto de ETH, que sigue aplicando.
PROBABILIDAD	Baja — exige una clave de agente comprometida y un token que acepte llamadas con valor.
REPORTADO POR	Pasada interna posterior, herramienta distinta

La severidad queda limitada por un detalle que el revisor no menciona: el `approve` del destino tiene que ser `payable` para que la llamada llegue a funcionar. Los ERC-20 estándar no lo son, así que el intento simplemente revierte; el test necesita un token payable fabricado a propósito para demostrar la vía. Explotarlo exige por tanto que el propietario haya whitelisteado un token inusual u hostil. Al lado hay una desprolijidad emparentada: un `transfer` que lleve `value` lo ejecuta el padre sin reenviar el ETH, y aun así el agente carga ese valor a su presupuesto de ETH. Ambas quedan ahora rechazadas: una llamada de token que lleve valor revierte con `TokenCallWithValue`, verificado por un test de regresión que además confirma que un `approve` normal, sin valor, sigue funcionando.

BVCC-12 **INFORMATIVA** **Mitigado por exclusión**

`increaseLiquidity` no comprueba el dueño en el position manager v3

UBICACIÓN	`NonfungiblePositionManager` de Uniswap v3; presets de call policy
IMPACTO	Alto si estuviera en la whitelist: a diferencia de `decreaseLiquidity`, `collect` y `burn`, el position manager no comprueba el dueño en `increaseLiquidity`. Un agente podría llevar los tokens aprobados de la wallet a una posición del atacante — un bypass de la lista de destinatarios que no se puede anclar, porque la palabra 0 es el `tokenId`, no una dirección.
PROBABILIDAD	No alcanzable tal como está desplegado: el selector está deliberadamente fuera de todos los presets de policy.
NOTA	Se registra para que nadie deshaga la exclusión añadiendo el selector por comodidad. Para añadir liquidez, el agente mintea una posición nueva con el destinatario anclado a la wallet. Reactivarlo exige un validator profundo que compruebe `ownerOf(tokenId) == wallet`. **Cerrado en V5:** es lo que hace el validator de Uniswap v3 de V5. `increaseLiquidity` solo se permite en una posición de la wallet y sin aprobación individual.

BVCC-13 **INFORMATIVA** **Aceptado**

Los constructores de los validators aceptan la dirección cero

UBICACIÓN	`BVCCUniversalRouterValidator.sol`, `BVCCPositionManagerValidator.sol` — constructores

IMPACTO	Ninguno tal como está desplegado. Un validator construido con router o position manager a cero no casaría con nada y denegaría todas las llamadas: un fallo en la dirección segura.
PROBABILIDAD	Solo error en el despliegue; no lo controla un atacante.
NOTA	Se releyó cada instancia desplegada en las seis redes y ambos inmutables son distintos de cero. Aceptado en vez de corregido: añadir la comprobación obligaría a redesplegar y volver a registrar validators que son demostrablemente correctos. **Cerrado en V5:** el constructor de todo validator de V5 revierte con la dirección cero.

Revisión de V5 — BVCC-14 a BVCC-30

Los hallazgos siguientes salen de las cinco rondas de V5 y del equipo rojo. Ninguno llegó a un contrato desplegado: la V5 se desplegó después de congelar el código el 2026-10-29. Cada uno se confirmó con un test que fallaba sobre el código de ese momento, la mayoría sobre un fork de una red real con los contratos de Uniswap y Aave desplegados. Esos tests pasan ahora como tests de regresión y tienen que seguir fallando con el código antiguo. Las garantías de V5 a las que se refieren (G-1 a G-7) están en el Anexo A.

BVCC-14 **ALTA** Corregido antes de desplegar

Una lectura de saldo fallida desactivaba el presupuesto de token del agente

UBICACIÓN	`BVCCAgentLogicV5.sol` — `prepare` / `settle` (medición)
IMPACTO	Un token permitido cuyo `balanceOf` revierte, o necesita más de las 100.000 unidades de gas de la lectura, salía de la wallet sin ningún cargo. Repitiendo envíos se podía vaciar.
PROBABILIDAD	Baja: necesita un token inusual, o uno cuya implementación cambie. Los ERC-20 estándar no se ven afectados.
ORIGEN	V5, ronda 1, `V5-H-01`. Rompe G-2.

La medición previa al lote guardaba si cada lectura de saldo había funcionado y seguía igualmente. Al liquidar, un token cuya lectura había fallado se trataba como si no hubiera salido nada. La prueba de concepto permitía una unidad de un token cuyo `balanceOf` revierte y enviaba diez: la transferencia pasaba y el presupuesto no registraba nada.

Arreglo. Una lectura fallida revierte ahora todo el lote con `MeasurementFailed` antes de ejecutar nada (`test_Measure_UnreadableBalanceReverts`). El coste se acepta y se documenta: un token así bloquea los lotes de ese agente hasta que el dueño lo quite de su lista.

BVCC-15 **ALTA** Corregido antes de desplegar

Una llamada a protocolo podía mover un token no permitido empaquetado en sus datos

UBICACIÓN	`BVCCAgentLogicV5.sol` — rama de protocolo y rastreo de los datos
IMPACTO	Usando una aprobación que el dueño había dado antes, un protocolo permitido podía sacar un token que el agente nunca tuvo permitido, sin pasar por ningún presupuesto.
PROBABILIDAD	Media: necesita una aprobación antigua del dueño y una función de protocolo que el dueño habilitó sin validator.
ORIGEN	V5, ronda 1, `V5-H-02`. Rompe G-2 y G-3.

La wallet encuentra los tokens a medir buscando en los datos de la llamada palabras completas de 32 bytes con direcciones. Un token empaquetado dentro de una ruta `bytes`, como los codifican los routers, es invisible para ese rastreo. En el primer borrador de V5 se podía permitir una función de protocolo sin validator, así que nadie más declaraba el token. La prueba de concepto usó un contrato con una ruta empaquetada y movió 100 unidades de un token fuera de la lista del agente.

Arreglo. Toda llamada de un agente a un protocolo necesita ahora un validator: una policy sin el bit `DEEP` revierte con `CallNeedsValidator`, sea cual sea la función. Desaparece la antigua lista de «funciones peligrosas». El validator responde de declarar todo token que la llamada pueda mover (ver BVCC-30). Como consecuencia, SwapRouter02 deja de estar disponible para agentes; sus swaps van por el Universal Router. Regresión: `test_Policy_EveryProtocolCallNeedsValidator`, más `test_Fork_UR_NotAllowedTokenRejected` en el router real.

BVCC-16 ALTA Corregido antes de desplegar

Una llamada a protocolo podía pagar a un destinatario fuera de la lista del agente

UBICACIÓN	`BVCCAgentLogicV5.sol` — rama de protocolo
IMPACTO	Una función de protocolo permitida que recibe un destino como argumento podía pagar a una dirección que eligiera el agente, fuera de su lista de destinatarios. El importe se cobraba, pero el destino no se comprobaba.
PROBABILIDAD	Media: necesita una policy de protocolo que el dueño habilitó sin ancla de destinatario.
ORIGEN	V5, ronda 1, `V5-H-03`, calificado «alto, condicional». Rompe G-4.

Los envíos directos comprobaban la lista de destinatarios; las llamadas a protocolos solo comprobaban las anclas que el dueño hubiera puesto en la policy, y el ancla de destinatario era opcional. El hallazgo dependía de cómo se leyera G-4, así que el equipo lo decidió: G-4 se aplica a todo pago que pueda provocar un agente.

Arreglo. El mismo que en BVCC-15. Con un validator obligatorio en toda llamada a protocolo, cada validator fija todos los destinatarios de pago de las funciones que permite (en la práctica, la propia wallet). Regresión: `test_Policy_EveryProtocolCallNeedsValidator` y `test_Fork_UR_ForeignRecipientRejected`.

BVCC-17 ALTA Corregido antes de desplegar

Retirar de Aave y reenviar salía a cero contra el presupuesto

UBICACIÓN	`BVCCAaveV3ValidatorV5.sol` — `withdraw`; `BVCCAgentLogicV5.sol` — liquidación
IMPACTO	Un agente podía convertir colateral de Aave en dinero y enviarlo fuera en un solo lote sin que su presupuesto registrara nada. Reproducido contra el Pool real de Aave v3: 99 USDC fuera con un límite de 1 USDC.
PROBABILIDAD	Media: necesita un agente con `withdraw` de Aave y colateral en la wallet. Nada más.
ORIGEN	V5, ronda 1, `V5-H-04`. Rompe G-2.

La V5 cobra la salida neta de cada token medido a lo largo del lote. El aToken no se medía, así que retirar 99 USDC a la wallet y después enviar 99 USDC dejaba el saldo de USDC donde empezó: salida neta cero, cargo cero, comisión cero, y el colateral perdido. El test sobre un fork de Arbitrum en el bloque 518.809.446 comprobaba exactamente eso.

Arreglo. Entradas cobradas. La interfaz del validator ganó un quinto valor de retorno, `chargeInflow`. Cuando un validator lo activa, la wallet lee los tokens declarados antes y después de esa llamada concreta y suma lo que entró a los costes del agente. Lo usan `withdraw` de Aave y toda llamada al PositionManager de Uniswap v4, así que sacar dinero de una posición que la wallet no mide se cobra al 0,15 % como cualquier otro gasto. Con el Pool real, retirar 99 USDC y reenviarlos con un límite de 10 revierte ahora, y retirar 5 cobra 5,0075 (`test_Fork_Aave_WithdrawIsChargedAndCapped`).

BVCC-18 **MEDIA** **Corregido antes de desplegar**

Pagar la comisión en un token con impuesto al emisor superaba el presupuesto

UBICACIÓN	`BVCCAgentLogicV5.sol` — pago de la comisión
IMPACTO	Con un token que cobra un impuesto al emisor en cada transferencia, la propia transferencia de la comisión costaba más de lo cargado, así que la wallet perdía algo más que el límite del agente.
PROBABILIDAD	Baja: necesita un token con impuesto al emisor en la lista del agente. El exceso es pequeño y depende del token.
ORIGEN	V5, ronda 1, `V5-M-01`. Rompe G-2 por una cantidad pequeña.

El cargo se anotaba antes de pagar la comisión y el saldo no se volvía a leer después. Con un impuesto del 10 % y un límite de exactamente 110,165 unidades, un envío de 100 le costaba a la wallet 110,1815.

Arreglo. Después de pagar cada comisión, la wallet vuelve a leer el saldo y revierte si bajó más que la comisión (`test_Fee_SenderTaxedTokenReverts`). Por tanto, los agentes no pueden mover tokens con impuesto al emisor; los que cobran al receptor sí funcionan. Figura entre los límites de V5 del §6.

BVCC-19 **MEDIA** **Aceptado — regla de despliegue**

Un validator mutable podía ampliar permisos sin la espera de 48 horas

UBICACIÓN	`BVCCValidatorRegistryV5.sol` — registro
IMPACTO	El registry retrasa 48 horas una *dirección* de validator nueva, pero llama al código que haya en esa dirección. Un validator con su propio admin, o detrás de un proxy, podría empezar a decir que sí al instante.
PROBABILIDAD	Baja tal como está desplegado: necesita que el admin del registry registre un validator mutable. Ninguno de los cinco validators desplegados tiene estado ni admin.
ORIGEN	V5, ronda 1, `V5-M-02`. Rompería G-6.

El registry no puede saber si el código al que apunta puede cambiar de opinión. Comprobar el hash del código no detectaría un cambio de estado ni la actualización de un proxy, así que no hay un arreglo de una línea en el contrato.

Decisión. G-6 lleva ahora una regla de despliegue: solo se registran validators inmutables y sin estado. Los cinco validators desplegados reciben todos sus parámetros en el constructor como inmutables y no guardan nada. Su código fuente está verificado en todos los exploradores, así que cualquiera puede comprobarlo antes de que pasen las 48 horas. Registrar un validator que incumpla la regla debilitaría G-6; por eso figura como supuesto de confianza aceptado y no como corregido.

BVCC-20 MEDIA Corregido antes de desplegar

No se comprobaba un hook congelado al añadir liquidez

UBICACIÓN	`BVCCPositionManagerValidatorV5.sol` — `INCREASE_LIQUIDITY`
IMPACTO	Después de que la gobernanza congelara un hook de Uniswap v4, un agente aún podía añadir liquidez a una posición existente en el pool de ese hook.
PROBABILIDAD	Baja: necesita un hook aprobado y congelado después, y una posición en su pool.
ORIGEN	V5, ronda 1, `V5-M-03`.

El validator comprobaba el hook en `MINT_POSITION` pero respondía que sí a `INCREASE_LIQUIDITY` sin buscar el pool de la posición.

Arreglo. El validator lee del PositionManager la clave del pool de la posición y exige que su hook sea ninguno o esté aprobado (`test_PM_IncreaseChecksHook`). La misma rama recibió después las comprobaciones de dueño de BVCC-21 y BVCC-22.

BVCC-21 ALTA Corregido antes de desplegar

Liquidez trasladada a una posición ajena por un cargo residual

UBICACIÓN	`BVCCPositionManagerValidatorV5.sol` — `INCREASE_LIQUIDITY`
IMPACTO	Un agente podía vaciar una posición de la wallet en una posición suya. Reproducido en Arbitrum con el PositionManager desplegado: se trasladó una liquidez de 1e12 mientras al agente se le cobraban como mucho dos unidades mínimas de cada token.
PROBABILIDAD	Media: necesita un agente que gestione liquidez v4 y una posición de la wallet. El resto lo prepara el propio agente.
ORIGEN	V5, ronda 2, `R2-H01`. Rompe G-2 y G-4.

El PositionManager de Uniswap v4 acepta acciones del dueño de una posición *o de un operador aprobado*. Un agente puede crear su propia posición y aprobar a la wallet como operadora. El validator solo comprobaba el hook en `INCREASE_LIQUIDITY`, así que una sola llamada a `modifyLiquidities` podía reducir la posición de la wallet y aumentar la del agente en el mismo pool y rango. El PositionManager compensa los dos movimientos por dentro, así que casi ningún ERC-20 salía de la wallet y la entrada cobrada (BVCC-17) no veía nada que cobrar. La especificación suponía que «Uniswap v4 ya exige ser el dueño»; esa nota era incorrecta.

Arreglo. `INCREASE_LIQUIDITY` exige `ownerOf(tokenId) == wallet`. Regresión en el fork de Arbitrum: `test_Fork_PM_ForeignOwnedNFTRejected` y `test_Fork_PM_LiquidityMoveToForeignNFTRejected`.

BVCC-22 ALTA Corregido antes de desplegar

Liquidez neteada hacia una posición propia aprobada a un tercero

UBICACIÓN	`BVCCPositionManagerValidatorV5.sol` — combinaciones de acciones
IMPACTO	Pérdida de una posición de la wallet: un agente pasaba su liquidez a una segunda posición de la wallet que el dueño había aprobado a otra persona, que después se la llevaba.
PROBABILIDAD	Media-baja: necesita una aprobación individual de NFT que el dueño diera antes sobre una de las posiciones de la wallet.
ORIGEN	V5, ronda 3, `R3-H01`. Rompe G-2 y G-4.

El arreglo de BVCC-21 comprueba que la posición de destino es de la wallet. No impide que esa posición esté aprobada a un tercero. En el test sobre fork, el dueño tiene dos posiciones en el mismo pool y rango y solo ha aprobado la pequeña a un tercero. El agente reduce la grande y aumenta la pequeña en una llamada; el tercero se lleva después la posición pequeña con la aprobación que ya tenía.

Arreglo. Dos reglas. Una llamada al PositionManager ya no puede sacar liquidez (`DECREASE`/`BURN`) y meterla (`MINT`/`INCREASE`) a la vez, en ningún orden, porque eso es lo que permite que la liquidez cambie de posición sin movimiento de tokens que cobrar. Y `INCREASE_LIQUIDITY` exige que la posición no tenga aprobación individual (`getApproved == 0`). Mover liquidez entre posiciones son ahora dos llamadas, y lo que sale paga el 0,15 % como entrada. Regresión: los tres tests de `AuditPositionManagerForkV5.t.sol`.

BVCC-23 ALTA Corregido antes de desplegar

Un hook v4 no aprobado movía un token no permitido por el Universal Router

UBICACIÓN	`BVCCUniversalRouterValidatorV5.sol` — `V4_SWAP` / `SWAP_EXACT_IN`
IMPACTO	Un agente sin ningún token permitido enviaba un swap por un pool con un hook elegido por él; el hook gastaba una aprobación antigua del dueño y se llevaba 1e18 unidades de un token no permitido.
PROBABILIDAD	Media-baja: necesita una aprobación antigua del dueño a la dirección del hook y acceso del agente al Universal Router.
ORIGEN	V5, ronda 3, `R3-H02`. Rompe G-2, G-3 y G-4.

El validator aceptaba `SWAP_EXACT_IN` sin decodificar su ruta, así que nunca veía el pool, el hook ni sus datos. Uniswap v4 ejecuta el `beforeSwap` del hook durante el swap, y un hook puede llamar a cualquier token con cualquier aprobación que la wallet haya dejado. El test sobre fork usó el Universal Router y el PoolManager desplegados en Arbitrum, con el hook en una dirección de prueba.

Arreglo. Se decodifica cada salto; se declaran para medir la moneda de entrada y todas las intermedias; se rechazan los tramos en nativo; y el hook de cada salto debe ser ninguno o estar aprobado en el hook registry. Un test extra sobre fork confirma que el validator lee la ruta igual que el router, incluido el Universal Router 2.1.2, que añade un campo al swap v4.

BVCC-24 ALTA Corregido antes de desplegar

Una retirada del agente ejecutaba un hook congelado

UBICACIÓN	`BVCCPositionManagerValidatorV5.sol` — `DECREASE_LIQUIDITY` / `BURN_POSITION`
IMPACTO	Un hook congelado por la gobernanza seguía ejecutándose cuando un agente retiraba liquidez, y podía gastar aprobaciones antiguas del dueño: 1e18 unidades de un token no permitido en el test sobre fork.
PROBABILIDAD	Baja: necesita un hook aprobado, congelado después por malicioso, y una aprobación antigua del dueño hacia él.
ORIGEN	V5, ronda 4, `R4-H01`. Rompe G-2 y G-3.

Retirar liquidez se dejó sin comprobar a propósito para que el dueño pudiera salir siempre de un pool con el hook congelado. Pero Uniswap v4 llama a `beforeRemoveLiquidity` también al retirar, así que un hook congelado seguía actuando cada vez que un agente retiraba.

Arreglo. Para agentes, `DECREASE` y `BURN` resuelven el pool de la posición y exigen que su hook sea ninguno o esté aprobado. Salir de un pool con un hook congelado es ahora decisión del

dueño, con su propia ejecución, que este validator no gobierna. Añadir, retirar y quemar declaran además las dos monedas del pool para medirlas. La contrapartida está en el §6: «siempre se puede salir» no está garantizado con un hook congelado, porque el hook puede seguir actuando o hacer fallar la salida.

BVCC-25 ALTA Aceptado — decisión de confianza

Un hook v4 aprobado ejecuta código que la wallet no puede acotar

UBICACIÓN	`BVCCUniversalRouterValidatorV5.sol`, `BVCCPositionManagerValidatorV5.sol` — comprobación de hooks
IMPACTO	Un hook aprobado malicioso o comprometido puede hacer lo que hizo el hook no aprobado de BVCC-23: gastar aprobaciones antiguas del dueño de tokens que el agente no puede usar.
PROBABILIDAD	Baja: necesita que la gobernanza apruebe un hook que sea, o se vuelva, hostil. Aprobarlo lleva 48 horas públicas y congelarlo es inmediato.
ORIGEN	V5, ronda 4, `R4-H02`; ya figuraba en la especificación como decisión de confianza.

Los validators comprueban la *dirección* del hook en el hook registry. No pueden limitar lo que hace su código, qué datos recibe ni qué aprobaciones usa. Fijar el hash del código detectaría un cambio de bytecode, pero no demostraría que el código es inofensivo, ni detectaría cambios de estado o la actualización de un proxy.

Decisión. Los hooks siguen disponibles para los agentes, así que G-2, G-3 y G-4 se cumplen con la condición de que los hooks aprobados sean de fiar. Cada hook se revisa antes de proponerlo, nunca se aprueban hooks actualizables (proxies), toda aprobación espera 48 horas a la vista y un hook se puede congelar al instante. Los agentes que solo usan pools sin hook no están expuestos. El dueño reduce más la exposición si no deja aprobaciones antiguas de tokens a contratos de hooks.

BVCC-26 ALTA Corregido antes de desplegar

Un agente podía vaciar la posición de un tercero que aprobó a la wallet

UBICACIÓN	`BVCCPositionManagerValidatorV5.sol` — `DECREASE_LIQUIDITY` / `BURN_POSITION`
IMPACTO	Pérdida total de la posición v4 *de un tercero*, NFT incluido en la variante con quema. La wallet se mantenía dentro de sus límites: los tokens retirados se cargaban al presupuesto del agente y luego se enviaban a un destinatario permitido.
PROBABILIDAD	Baja: necesita un tercero que haya aprobado a la wallet como operadora de su posición.
ORIGEN	Equipo rojo, `RT-01`. Reproducido sobre forks de cinco redes, contra cinco versiones desplegadas del PositionManager.

Es BVCC-21 al revés. La comprobación de dueño se había añadido solo a `INCREASE_LIQUIDITY`, y el PositionManager acepta operadores aprobados en todas las acciones, así que un agente podía retirar y quemar una posición que no era de la wallet.

Arreglo. `DECREASE` y `BURN` exigen también `ownerOf(tokenId) == wallet`. Los forks de cinco redes rechazan la posición ajena en ambas acciones. La ronda 5 pidió tests de retirar y quemar la misma posición en una llamada, en los dos órdenes, propia y ajena; se añadieron, incluido uno con estado real de Arbitrum.

BVCC-27 MEDIA Corregido antes de desplegar

Los agentes no podían quemar nunca una posición v4

UBICACIÓN	`BVCCAgentLogicV5.sol` — rastreo de los datos
IMPACTO	Denegación de servicio, sin robo: un agente podía abrir posiciones v4 pero toda quema revertía, así que nunca podía cerrar una.
PROBABILIDAD	Segura para cualquier agente que gestione liquidez v4.
ORIGEN	Equipo rojo, `RT-55`.

El rastreo trata como token a cualquier contrato que responda a `balanceOf`. El PositionManager es un ERC-721 y responde con el número de posiciones de la wallet, así que quemar una parecía la salida de un token no permitido y el lote revertía con `TokenNotAllowed`. La misma quema hecha por el dueño funcionaba.

Arreglo. El rastreo descarta los contratos que se declaran ERC-721 por ERC-165, con una consulta de solo lectura y gas limitado. Los tokens de la lista del agente y los declarados por validators se siguen midiendo primero. La ronda 5 comprobó que un falso «ERC-721» no puede usar esto para esconder un token permitido o declarado.

BVCC-28 BAJA Corregido antes de desplegar

`pauseAgents` revertía si la wallet ya estaba en pausa

UBICACIÓN	`BVCCAgentWalletV5.sol` — `pauseAgents` / `unpauseAgents`
IMPACTO	Un lote del dueño que pausaba a todos los agentes fallaba si ya estaban en pausa, por ejemplo tras una recuperación. Sin riesgo para los fondos.
PROBABILIDAD	Uso normal del dueño.
ORIGEN	Equipo rojo, `RT-30-A`.

Arreglo. Las dos funciones no hacen nada si la wallet ya está en el estado pedido. Los eventos se siguen emitiendo, así que quien vigile la wallet debe leer `paused()` para conocer el estado.

BVCC-29 MEDIA Aceptado — supuesto de confianza

Un token permitido que miente sobre su saldo o mueve otro token

UBICACIÓN	Contratos de token en la lista del agente
IMPACTO	Un token permitido malicioso puede salir sin cargo declarando un saldo falso, o sacar otro token distinto de la wallet al transferirse.
PROBABILIDAD	Baja: el dueño tiene que poner un token así en la lista del agente.
ORIGEN	Equipo rojo, `RT-52` (bajo) y `RT-54` (medio), unidos aquí.

La V5 mide saldos, y un saldo es lo que diga el contrato del token. Un token también puede ejecutar cualquier código en `transfer`, incluida la transferencia de un segundo token que tenga permiso para mover. Ninguna wallet puede defenderse de un token que miente sobre sí mismo.

Decisión. Los tokens permitidos son de confianza, igual que un hook aprobado: elegirlos es decisión del dueño. La lista de un agente debe contener ERC-20 estándar. El caso del impuesto en el mismo token falla cerrado (BVCC-18).

BVCC-30 BAJA Aceptado — regla de validators

La wallet solo mide los tokens que ve o que declara un validator

UBICACIÓN	`BVCCAgentLogicV5.sol` — rastreo de los datos; todos los validators
IMPACTO	Un token empaquetado en los datos y no declarado por el validator de la llamada no se mide. Con un validator que declare de menos, podría salir sin presupuesto.
PROBABILIDAD	Baja con los validators desplegados, que declaran los tokens de cada acción que aceptan. Sería media si un validator registrado declarara de menos.
ORIGEN	Equipo rojo, `RT-65`.

Es lo que queda de BVCC-15. El rastreo lee palabras completas de 32 bytes; una dirección empaquetada es invisible para él, igual que un token cuya lectura de saldo necesita más gas del tope y no está en la lista permitida.

Regla. Un validator debe declarar todo token que su contrato destino pueda mover y rechazar lo que no sepa decodificar. El validator del Universal Router declara todas las monedas de la ruta, los de posiciones declaran las dos monedas del pool, el de Aave declara el activo y el del nativo envuelto declara el token envuelto.

5 Remediación y verificación del despliegue

La V5 es la generación que está viva hoy, en seis redes. Esta sección recoge qué está desplegado, cómo se comprobó al margen de los scripts de despliegue y qué se ha ejercitado en redes reales. Después sigue el registro de la remediación de V4, como generación anterior.

Despliegue de V5 — Ethereum · Arbitrum One · Base · BNB Chain · Polygon · Arbitrum Sepolia

CONTRATO	DIRECCIÓN (CREATE2, IDÉNTICA EN TODAS LAS REDES)
Librería de lógica de agentes (`BVCCAgentLogicV5`)	`0x7504bf3c1230eF99eaEFAc03287c8F036CBb6D0E`
ValidatorRegistry V5	`0x7269334AE74DCE4e607eec36268383DD35f28350`
HookRegistry V5	`0xEaEaC7F6071f7F576Be2aD97898Dd0c0F51BE78A`
Lógica SmartWallet V5	`0xc9dE89d54beB786bc916131CA6c8c5374507Bd15`
Lógica AgentWallet V5	`0x360f991B23a3D5202Dc422292697aB07dA279cFA`
SmartWalletFactory V5	`0x2672951DC1E3f865BA6Fc98e5509dc4B23B260a2`
AgentWalletFactory V5	`0xE36fb66c21C38536454C1a58fBC5EB1Cb455aD52`

Todo se desplegó con el desplegador CREATE2 estándar (`0x4e59…956C`): Arbitrum Sepolia el 2026-10-30 y las cinco mainnets el 2026-11-02, con el código congelado el 2026-10-29. El script se niega a desplegar en una mainnet si las dos factories no caen en las direcciones fijadas de arriba. Las wallets son clones ERC-1167 de los dos contratos de lógica y no se pueden actualizar. Los registries y el interruptor `kill` de las factories pertenecen a la hardware wallet de gobernanza `0x3145Bd5e…5A4c`, con la propiedad transferible solo en dos pasos, y las comisiones van a `0x3e3eb089…07D4`, una constante compilada en la lógica. Ninguna de las dos direcciones puede mover fondos de usuarios: los registries no guardan nada y `kill` solo impide crear wallets nuevas.

Crear una wallet cuesta unas 537.000 unidades de gas, frente a unos 4,7 millones en V4, porque cada wallet es un clon pequeño. La lógica de la wallet de agentes ocupa 22.604 bytes, 1.972 por debajo del límite EIP-170, con el código de permisos movido a la librería externa.

Validators de V5 por red

Cada validator está ligado a un contrato de protocolo, así que la mayoría de direcciones cambian por red. Los 30 se releyeron con `validators(target)` en el registry de V5 el 2026-11-11: todos están **activos** y apuntan al validator esperado. Los tres primeros de Arbitrum Sepolia se activaron el 2026-11-01 y los otros 27 entre el 2026-11-04 y el 2026-11-08. Todos pasaron por la espera pública de 48 horas del registry.

RED	VALIDATOR DE	DIRECCIÓN
Ethereum	Universal Router	`0x48A6d5415C369Fa38E0ebEafC779527fb6cF32A0`
Ethereum	PositionManager de Uniswap v4	`0xb05fc3aaD6E4cAaD69b680C1adae645DB0B56f1f`
Ethereum	Pool de Aave v3	`0xeD51c5063dB3bB4609Cf95fC349d87b68bEECC28`

Ethereum	Posiciones de Uniswap v3	`0xfcf4AB3E1FC5b32A00161e66ebe537B617fC9678`
Ethereum	Nativo envuelto	`0xDC2480bD3DfDC5A4c92ab1C37f2234FF5f91521F`
Arbitrum One	Universal Router	`0x332E88FDc5f26a7421534902Cbb9C95B70E8AA0d`
Arbitrum One	PositionManager de Uniswap v4	`0xc9Ae38709cd76b51F4f11a21Ee2aAC1cf67a013D`
Arbitrum One	Pool de Aave v3	`0xC7bD86E147b211DDC1c6bbCad07A01B51C7CAff4`
Arbitrum One	Posiciones de Uniswap v3	`0xfcf4AB3E1FC5b32A00161e66ebe537B617fC9678`
Arbitrum One	Nativo envuelto	`0x80DBf05b4406927c22cDFaFb3f226665F863A58D`
Base	Universal Router	`0x5b9cDCC1a4Ac139E8b13A4FC4ca4519906F651A5`
Base	PositionManager de Uniswap v4	`0x0E43866CFFC41E814C1f4e0341Fed09234d1d8B1`
Base	Pool de Aave v3	`0x30Bd59CF41c2F4FE02e9666065dd294FD44333bb`
Base	Posiciones de Uniswap v3	`0x2B52943E9F361CCA42f1100a78b74daaEB25F02e`
Base	Nativo envuelto	`0xEF913B99C970E4Bc3B987F0668d0F20CD8B90c6E`
BNB Chain	Universal Router	`0x656f0EB67ce76c3b416D8F3dFB46e81c7aA442D7`
BNB Chain	PositionManager de Uniswap v4	`0x12298d84D1d3272a51a7b21755D222da26996786`
BNB Chain	Pool de Aave v3	`0x822E3ab17d7AC4E49cA18E881392CED15ba1E7bb`
BNB Chain	Posiciones de Uniswap v3	`0x5C700655e00FA7003672617d8723d5AeBF9E9c9A`
BNB Chain	Nativo envuelto	`0x4F2974859169262158065a1506070ee52EFeD6De`
Polygon	Universal Router	`0xDd313C3884a6fA1636fc5cba3E251874073e1B3f`
Polygon	PositionManager de Uniswap v4	`0xd5315B0B30Fa25C5EcbFAA2BBD514262E33Db911`
Polygon	Pool de Aave v3	`0xC7bD86E147b211DDC1c6bbCad07A01B51C7CAff4`
Polygon	Posiciones de Uniswap v3	`0xfcf4AB3E1FC5b32A00161e66ebe537B617fC9678`
Polygon	Nativo envuelto	`0xc4E512BEEbBCb03A79C10c109B4CD6cef8aa5E5e`
Arbitrum Sepolia	Universal Router	`0x1D0394e997a0799883AE433383Bc82280E17E403`
Arbitrum Sepolia	PositionManager de Uniswap v4	`0x6Cd6e2D7328b0128Ab82921939526dde6f45F2d3`
Arbitrum Sepolia	Pool de Aave v3	`0xA61ac81B9fAA48eE33Ac8A1f28ad80C577e276FA`
Arbitrum Sepolia	Posiciones de Uniswap v3	`0xbaBA421723a25363BB25468B94A1100B3972DB84`
Arbitrum Sepolia	Nativo envuelto	`0x0A9F4027e3A2136d4e688D0Ee0A5b42cE94b3702`

> **72 de 72 contratos con el código verificado.** Doce contratos por red (los siete del núcleo y cinco validators), comprobados desde fuera con la API de los exploradores el 2026-11-11: todos verificados, en las seis redes. Cualquiera puede comparar el código publicado con el árbol citado en el §2.

V5 en redes reales

DÓNDE	QUÉ SE EJECUTÓ	RESULTADO
Arbitrum Sepolia,	Un agente manejado con el servidor MCP sobre una wallet de agentes V5: swaps en Uniswap v3 y v4 y desde	Todo según el diseño. Las operaciones por encima del

2026-11-01 → 11-08	nativo, depositar, pedir prestado, devolver y retirar en Aave, posiciones de Uniswap v4 y v3 abiertas, cobradas, reducidas, cerradas y quemadas, envolver y desenvolver, envíos, y cada límite probado a propósito	límite, a destinatarios ajenos o no soportadas se rechazaron con el error esperado antes o al ejecutar
Arbitrum Sepolia, 2026-11-06	Recuperación por guardianes: dos guardianes votaron la misma passkey nueva, la propuesta se armó y 48 horas después se ejecutó	Clave y credencial del dueño rotadas; todos los agentes en pausa, como está especificado; el dueño los reactivó con la passkey nueva
BNB Chain, 2026-11-11	Migración de una wallet de agentes V4 a una wallet de agentes V5 (`0x566669C6…4936`) con «Enviar todo»: una firma del dueño, un lote	`0x549492c6…91724b` (BNB) y `0xf0141064…ac27a16` (USDC y BNB juntos): importes y comisión del 0,15 % exactos

> Antes del despliegue, el despliegue de producción se simuló también sobre forks de las seis redes con los monederos reales de comisiones y admin, incluida una operación del dueño por el EntryPoint real (v0.9) y un agente autorizado por UserOp y operando. **Lo que aún no se ha ejecutado** es un agente operando una wallet V5 en una mainnet. La vía del agente está cubierta en Arbitrum Sepolia y en forks de mainnet; lo que este informe afirma para mainnet es «desplegado, verificado y conectado», no «ejercitado por un agente».

Remediación de V4 (generación anterior)

Nueve hallazgos de la revisión de V4 están remediados en V4. Siete exigían cambiar el bytecode y viajaron juntos como la generación V4; los otros dos se cerraron antes de que el código afectado llegara a ninguna red. El resto de esta sección recoge qué está vivo para V4, dónde, y cómo se comprobó.

Siete hallazgos exigían cambiar el bytecode. Los siete están corregidos y **vivos en las seis redes**. Como las direcciones CREATE2 se derivan del bytecode, no se podían aplicar a las wallets ya desplegadas: la migración es el mismo playbook que V1→V2 y V2→V3 —los usuarios recrean su wallet y mueven fondos, tras lo cual se hace `kill()` de las factories sustituidas—. Viajaron en un solo despliegue en vez de forzar varias migraciones. Los contratos se renombraron a V4 y el dominio EIP-712 va detrás (`BVCCSmartWalletV4`), igual que en la generación V2→V3; la interfaz lee ese dominio para detectar una wallet obsoleta y ofrecer la migración.

ELEMENTO	ESTADO
Reentrada cruzada (BVCC-01)	**Corregido** dos capas, cada una con su test de regresión
Apropiación de guardianes (BVCC-04)	**Corregido** la factory ya no los elige; `setGuardians` exige passkey
`credentialId` sin autenticar (BVCC-06)	**Corregido** fuera de la factory, a un evento firmado que emite la wallet
Credencial obsoleta tras recuperación (BVCC-07)	**Corregido** `setCredentialId`, con auto-llamada
`approve` con ETH (BVCC-11)	**Corregido** las llamadas de token deben llevar valor cero
Los agentes sobreviven a la recuperación (BVCC-05)	**Corregido** la recuperación pausa los agentes
Los guardianes no se podían rotar (BVCC-08)	**Corregido** el dueño puede reemplazarlos, nunca durante una recuperación

Contabilidad de token en caso 3 (BVCC-03)	**Abierto** escalonado; la fase de validators es la que lo cierra
Bloqueo de recuperación por un guardián (BVCC-09)	**Abierto** aceptado; denegación de servicio, no robo
Zero-check en constructores de validators (BVCC-13)	**Abierto** aceptado; inerte — se releyó cada instancia desplegada y ambos inmutables son distintos de cero
UX de revocar allowances y defaults de topes (BVCC-10)	**Abierto** trabajo de interfaz — deliberadamente fuera del contrato

Despliegue de V4 — Arbitrum One · Ethereum · BNB Chain · Base · Polygon · Arbitrum Sepolia

CONTRATO	DIRECCIÓN (CREATE2, IDÉNTICA EN TODAS LAS REDES)
SmartWalletFactory V4 (salt …0A)	`0xfd105197109244483b5f870501326E6faec9F93c`
AgentWalletFactory V4 (salt …0B)	`0xf3A61F9d64d45362E149A111289546523BCd26a6`
ValidatorRegistry	`0x5e371D54AC97a57B0a99145Ed04A3c9fA07850C2` — sin cambios

Verificado al margen del script de despliegue, releyendo cada red: el bytecode desplegado es idéntico en las seis (smart wallet 15.959 bytes, agent wallet 24.541), el owner es la wallet hardware de gobernanza `0x3145Bd5e…A4c` y el registry está intacto. Las factories están verificadas en los exploradores.

El registry queda deliberadamente intacto: es la dirección que va compilada como constante dentro de cada wallet, y el test de congelación demuestra que la wallet V4 sigue resolviendo a ella. Los validators se comparten entre generaciones, así que no hubo que volver a registrar nada para V4.

Liquidez en v4 — ciclo completo ejecutado por un agente en mainnet

La ruta de validación profunda que introdujo V3 y hereda V4 se recorrió entera con un agente en Arbitrum One, contra un pool nativo: el caso que el anclaje por bitmap no sabe expresar, y la razón de ser de `BVCCPositionManagerValidator`. Pool ETH/USDC, fee 500, tick spacing 10, sin hooks, rango completo.

PASO	TRANSACCIÓN	GAS	RESULTADO
mint	`0xb4245927…f1269`	622.967	tokenId 190877, `ownerOf` = wallet
lectura de la posición	—	—	currency0 nativo, currency1 USDC, liquidez 4.377.897.532
collect	`0x9296f6f3…a865c7`	218.749	correcto, cero proceeds, cero fee
decrease 100% + burn	`0x92cce43d…dca9ef`	267.739	NFT quemado

El validator se comportó como estaba diseñado en todo el ciclo: las acciones `TAKE_PAIR` y `SWEEP` se aceptaron con `recipient` resolviendo a la wallet, y el dry-run pasó en cuanto el PositionManager entró en `allowedProtocols`. El mint envió 0,0001 ETH y el `SWEEP` devolvió los 17.050 wei de sobrante nativo, de modo que el PositionManager se quedó con **cero ETH**: justo el caso de valor varado que el validator rechaza cuando ningún `SETTLE` nativo consume el value. Estado final releído en cadena para este informe: `ownerOf(190877)` revierte `NOT_MINTED`, la liquidez de la posición es 0 y el saldo nativo del PositionManager es 0.

> Este ciclo se ejecutó **solo en Arbitrum One**. En las otras cinco redes los validators están desplegados, registrados y confirmados como atados al PositionManager correcto, pero el ciclo de liquidez de extremo a extremo no se ha ejecutado. Lo que afirma este informe es, por tanto: probado de extremo a extremo en Arbitrum, desplegado y verificado idéntico en el resto.

La factory de agentes queda en **24.541 bytes**, 35 por debajo del límite EIP-170, con `executeAsAgent` en caso 3 a 64.972 gas. Ese margen es estrecho, y es la razón de que dos medidas de endurecimiento candidatas se dejaran deliberadamente fuera del contrato: exigir topes de token no nulos cuando un agente tiene acceso a protocolos, y limitar las aprobaciones del propio dueño mientras haya un agente activo. Las dos pertenecen a la interfaz, donde no cuestan nada y pueden explicarse. El bytecode restante queda reservado para medidas sin equivalente fuera de la cadena.

6 Riesgo residual y recomendaciones

En V5 no hay nada abierto. Cuatro hallazgos se aceptan como supuestos de confianza, la V5 tiene límites conocidos que el dueño debe entender, y las wallets que siguen en V4 o anteriores conservan sus propios hallazgos abiertos hasta que sus dueños migren. Esta sección dice qué debería hacer un dueño.

Lo primero: migrar

Una wallet desplegada no se puede actualizar por dentro. Una wallet V4 sigue con **BVCC-03** abierto: si el dueño ha dado una allowance a un protocolo que un agente puede alcanzar, una clave de agente comprometida vale más que su presupuesto. Las wallets de V1 a V3 además ejecutan el crítico BVCC-01. La solución es crear una wallet V5 con la misma passkey y pasar los fondos. La app detecta una wallet antigua, enumera lo que cambia en V5 y ofrece una guía de migración y un botón «Enviar todo» que mueve todos los tokens elegidos de una red con una firma. Las posiciones en Uniswap o Aave pertenecen a la dirección antigua y hay que cerrarlas antes; los agentes hay que autorizarlos de nuevo en la wallet nueva y revocarlos en la antigua.

Qué pide la V5 que el dueño confíe

- **Hooks de Uniswap v4 aprobados (BVCC-25).** Un hook aprobado ejecuta su propio código en los swaps y en la liquidez de los agentes y puede alcanzar aprobaciones antiguas del dueño. Toda aprobación de hook espera 48 horas a la vista y se puede congelar al instante; los agentes que se limitan a pools sin hook no están expuestos.
- **Tokens permitidos (BVCC-29).** Un token que miente sobre su saldo, o que mueve otro token al transferirse, derrota cualquier límite basado en saldos. Da a los agentes solo ERC-20 estándar.
- **Validators (BVCC-19, BVCC-30).** Solo se registran validators inmutables y sin estado, y cada uno debe declarar todo token que deje mover. Los cinco validators desplegados cumplen las dos reglas, y su código está verificado en todas las redes.
- **Las aprobaciones del propio dueño.** Las allowances que el dueño da desde cualquier dApp viven en los contratos de los tokens. Un agente no puede usarlas por encima de su presupuesto, pero un hook aprobado hostil sí podría (BVCC-25), así que revoca las que ya no uses.

Límites conocidos de V5

- **No es actualizable.** Un fallo grave exigiría una V6 y otra migración.
- **Los agentes deben ser EOAs normales**, comprobado al autorizar y en cada llamada. Una dirección de agente con una delegación EIP-7702 deja de funcionar.
- **Los límites significan lo que dicen.** 0 no permite nada, una lista de destinatarios vacía no permite a nadie, y «sin límite» hay que elegirlo de forma explícita. Los permisos de un agente caducan en 365 días como mucho y se pueden renovar.
- **La comisión se cobra encima**, el 0,15 % de lo que sale de la wallet, dentro de los límites del agente. Un agente no puede enviar todo su saldo de un token: tiene que dejar sitio para la comisión.
- **Los tokens con impuesto al emisor** no los pueden mover los agentes (BVCC-18). Los que cobran al receptor funcionan.
- **Con un hook congelado**, solo el dueño puede intentar salir de ese pool, y el hook puede seguir actuando o hacer fallar la salida (BVCC-24).

- **Mover liquidez entre posiciones** son dos llamadas, y lo que sale paga el 0,15 % como entrada (BVCC-22). La parte en ETH nativo de salir de una posición v4 no se cobra al entrar.
- **La recuperación es por red.** Tras perder la passkey original, una wallet no se puede recuperar en una red donde nunca se desplegó: nacería con la clave perdida y sin guardianes. Despliega en todas las redes que puedas usar.
- **Los agentes siguen operando mientras hay una recuperación armada** y se pausan al ejecutarla. Fue una decisión deliberada: una recuperación significa una passkey perdida, no robada, y pausar 48 horas podría impedir que un agente gestione un préstamo de Aave.
- **Los NFT no se miden.** Los agentes solo llegan a las posiciones a través de validators, y lo que sale de una posición hacia la wallet se cobra.

Recomendaciones operativas

1. **Migra** toda wallet V4 o anterior que tenga fondos, en todas las redes.
2. **Despliega en todas las redes soportadas**, para que la dirección sea tuya en todas y la recuperación funcione allí.
3. **Da a cada agente lo mínimo que necesite**: tokens, límites, destinatarios y caducidad concretos. Pausa solo a un agente en vez de a todos cuando solo uno se porte mal.
4. **Revoca las aprobaciones de tokens que ya no uses**, sobre todo las dadas a contratos de hooks.
5. **Vigila los registries.** Cualquier ampliación de lo que pueden hacer los agentes se anuncia en cadena 48 horas antes de tener efecto; pausar o revocar agentes en esa ventana es inmediato.

Wallets que siguen en V4 o anteriores

Hasta que migren, estos dueños conservan el cuadro de V4. Anclar el destino no es lo mismo que acotar el valor: donde existe una allowance viva, una clave de agente comprometida puede cambiar un activo valioso por uno sin valor en un pool que controla el atacante, y el valor se queda con él. La medida provisional no necesita cambiar el contrato: **pon a cero toda allowance viva hacia los protocolos que un agente pueda alcanzar antes de autorizarlo**, y prefiere aprobaciones por operación. BVCC-09 (un solo guardián puede bloquear una recuperación) y BVCC-10 (pausar un agente no retira sus allowances) también siguen aplicando allí. Los fondos que queden en V1, V2 o V3 ejecutan bytecode con BVCC-01 sin corregir; en esas wallets, una lista de destinatarios no vacía que excluya la dirección del propio agente es lo que lo cierra.

7 Conclusión y limitaciones

V5. La generación V5 cambia «comprobar adónde puede ir el dinero» por «medir lo que de verdad ha salido». Ese cambio cierra BVCC-03, el único riesgo material que dejó abierto V4, y con él la clase de problemas en los que una allowance viva o un lote bien combinado dejaba a un agente gastar fuera de su presupuesto. La V5 conserva además todos los arreglos de V4, cierra el bloqueo de recuperación de BVCC-09, impide que el agente deje allowances vivas, hace que 0 signifique nada y que todo permiso caduque, y exige un validator en toda llamada a protocolo. Cinco rondas de revisión y un equipo rojo encontraron diecisiete problemas en el código nuevo antes de publicarlo: trece corregidos y cuatro aceptados como supuestos de confianza declarados. La última ronda no encontró nada nuevo. La V5 está desplegada en seis redes con los 72 contratos verificados y los 30 validators activos.

La mayoría de los hallazgos de V5 salieron del mismo sitio: los validators de posiciones de Uniswap, donde el valor puede moverse entre NFTs sin que ningún token salga de la wallet. Cada arreglo ahí vino seguido de un hallazgo a su lado, hasta que las reglas se hicieron simétricas (comprobación de dueño en todas las acciones, nada de sacar y meter en una llamada, hooks comprobados en todos los caminos). Ese patrón es la razón principal para seguir revisando la V5, y el Anexo G lo recoge.

Generaciones anteriores. La BVCC Agent Wallet aplica un modelo de permisos completo y por capas. Se identificaron tres incidencias de severidad alta a lo largo de tres rondas de revisión, cada una reproducida, remediada, redesplegada y re-verificada, y la batería de permisos on-chain pasa sobre el bytecode parcheado. Al preparar esta edición aparecieron cuatro más: una reentrada cruzada, la apropiación de guardianes por la factory, el gasto sin acotar del caso 3 (BVCC-03) y los agentes sobreviviendo a una recuperación. Todas menos una están corregidas y desplegadas en la generación V4 de factories, junto con los arreglos de la credencial, la rotación de guardianes y el `approve` con valor. La recuperación por guardianes se ejecutó y verificó de extremo a extremo —rotación de propiedad a una nueva passkey y rechazo on-chain de la clave antigua (ver Anexo D). La ronda V2 corrigió un problema de gas en mainnet en el snapshot de comisiones y concluyó con un despliegue multichain determinista (ver Anexo D).

La ronda V3 es el cambio más profundo hasta ahora en la vía del agente. Cerró un vector de exfiltración que los presupuestos no podían ver, haciendo las llamadas DeFi del agente default-deny por selector y anclando el destino o bien a la propia wallet o bien a un validator on-chain cuyo registro es asimétrico: inmediato para denegar, 48 horas para permitir (ver Anexo A y Anexo A). Dentro de ese trabajo se cazaron dos bypasses en potencia: una máscara de comandos que habría aceptado un depósito en un puente cross-chain como si fuera un swap, y una vía `increaseLiquidity` sin comprobación de propiedad, que se dejó deliberadamente sin registrar. El modelo está verificado por 264 tests unitarios, de fork y de fuzz, por un swap real en mainnet en el que todos los destinatarios resuelven a la wallet y la comisión es exacta, y por el rechazo de la variante adversarial de ese mismo swap antes de ejecutarse (ver Anexo B).

V4 cerró esa ronda y está vivo en las seis redes, con la capa de validación profunda completa: los dos selectores que delegan en el registry —el `execute` del Universal Router y el `modifyLiquidities` del PositionManager v4— tienen validator activo en todas partes, y el ciclo de liquidez completo se ejecutó con un agente en mainnet de Arbitrum, terminando con la posición quemada y sin valor nativo varado en el PositionManager (ver Anexo A y Anexo D).

El riesgo residual se expone sin adornos. Las llamadas a protocolo (caso 3) son seguras en el sentido que V3 se propuso: el destino queda anclado. Pero anclar el destino no es lo mismo que acotar el valor, y donde exista una allowance viva esa diferencia es la pérdida entera. Ese hueco sobrevive a V4: una clave de agente comprometida sobre una wallet dejada en el default permisivo,

con una allowance ya concedida, sigue valiendo más que su presupuesto. El plan escalonado del BVCC-03 dice qué puede estrechar la interfaz ahora y qué necesita la capa de validators para cerrarse de verdad. Aparte, todos los hallazgos de este informe siguen aplicando a los fondos que estén en wallets V1, V2 o V3, porque una wallet desplegada no se puede actualizar en su sitio: la exposición termina cuando el usuario migra, no cuando sale V4.

Qué establece y qué no establece esta revisión

Establece que los hallazgos concretos aquí recogidos se reprodujeron, se remediaron donde así se indica, y que la remediación está presente en bytecode desplegado en las redes del Anexo F — cada una releída de la cadena, no deducida de un script de despliegue. Cada hallazgo tiene su test de regresión y la suite pasa. Para V5 establece además que los contratos desplegados tienen el código verificado (72 de 72) y que los 30 validators están activos y apuntan donde deben.

No establece que los contratos estén libres de otros defectos. Es una revisión interna hecha por el equipo que escribió el código. Los cuatro hallazgos aportados por una pasada posterior con otra herramienta —BVCC-04, BVCC-06, BVCC-07 y BVCC-11— habían sobrevivido cada uno a tres rondas anteriores, que es la medida más clara disponible de lo que se deja repetir el mismo método. **Ningún tercero independiente ha revisado este código**, así que nada de lo que hay aquí lo ha comprobado alguien con interés en encontrar fallos. Los contratos no han sido auditados externamente y este documento no debe leerse como sustituto de una auditoría.

Las rondas de V5 se hicieron con herramientas de análisis con IA, dirigidas por el mismo equipo. Encontraron más antes del despliegue que ninguna ronda anterior, y también mostraron su límite: las rondas 2 a 4 encontraron cada una un camino que la anterior no había buscado. El auditor de la quinta ronda no pudo comparar por su cuenta el código con el de la ronda 4; esa comparación la hizo el equipo. Una revisión con IA es un buen segundo par de ojos, pero no es una auditoría independiente.

La cobertura tampoco es uniforme por red. En V4, la batería de permisos se ejecutó en Arbitrum Sepolia; la capa de call policies y el ciclo de liquidez de Uniswap v4, en Arbitrum One. En las otras cuatro redes los contratos están desplegados y verificados byte a byte, y se releyó el cableado de los validators, pero no se ejecutó ningún ciclo de extremo a extremo. En V5, la batería completa del agente se ejecutó en Arbitrum Sepolia y en forks de mainnet; en mainnet se han hecho operaciones del dueño (BNB Chain), pero ningún agente ha operado todavía una wallet V5.

A Arquitectura de seguridad

Material de referencia sobre los mecanismos a los que aluden los hallazgos. Primero va el diseño de V5, porque es la generación actual; después la capa de call policies y el registry de validators de V3 y V4, en los que se apoya la V5 y que siguen usando las wallets V4.

V5 — qué garantiza

La especificación de V5 declara siete garantías. Cada una es un invariante que comprueba la suite de tests, varias con 128.000 secuencias aleatorias de acciones.

ID	GARANTÍA
`G-1`	Solo la passkey del dueño, o una recuperación 2-de-3 de los guardianes completada tras 48 horas, puede cambiar quién es el dueño.
`G-2`	Un agente nunca saca de la wallet más de un token que su presupuesto para ese token, comisión incluida, haya las aprobaciones que haya y combine como combine las llamadas.
`G-3`	Un agente nunca mueve un token que no tenga permitido.
`G-4`	Un agente nunca envía fondos a un destino que no tenga permitido, directamente o a través de un protocolo.
`G-5`	Un agente no puede llamar a las funciones de la propia wallet ni recuperar el control de dueño a mitad de operación (reentrada).
`G-6`	La gobernanza externa solo puede restringir al instante; ampliar lleva 48 horas visibles en cadena. Exige validators inmutables y sin estado (BVCC-19).
`G-7`	Nadie salvo el dueño fija la credencial o los guardianes de la wallet, tampoco quien la despliega.

G-2, G-3 y G-4 se cumplen con la condición de que los hooks aprobados y los tokens permitidos sean de fiar (BVCC-25, BVCC-29).

V5 — el lote del agente, paso a paso

El agente envía su propia transacción a `executeAsAgent(mode, executionData)`; solo se acepta el modo de lote ERC-7579. Dentro:

1. **Comprobación del agente.** Una EOA normal en esta misma llamada, activa, sin caducar, sin pausa global y sin pausa propia.
2. **Por operación, antes de ejecutar nada.** El destino no es la wallet ni está bloqueado por el dueño. Un envío nativo debe ir a un destinatario permitido; un `transfer`, de un token permitido a un destinatario permitido; un `approve`, de un token permitido a un *protocolo permitido*. Cualquier otra llamada necesita un protocolo permitido, una policy para ese agente y función con el bit `DEEP`, las anclas que haya puesto el dueño y un sí del validator del protocolo, que además devuelve los tokens a medir, los costes (como un préstamo), si quiere una comprobación posterior y si se cobran las entradas.
3. **Medir.** Leer el saldo de todo token permitido, todo token encontrado en los datos y todo token declarado por un validator, hasta 60. Una lectura fallida revierte.
4. **Nativo.** Cargar el valor nativo enviado, más su comisión, a los límites nativos.
5. **Ejecutar** el lote con la marca de reentrada puesta, midiendo las entradas cobradas alrededor de las llamadas que lo pidieron.
6. **Restaurar aprobaciones.** Cada aprobación que hizo el agente vuelve a su valor anterior.

7. **Medir otra vez y cobrar.** Por cada token que bajó: uno no permitido revierte el lote; uno permitido se carga salida + costes + comisión del 0,15 % contra los límites por lote, diario y total, y la comisión se paga en ese mismo token.
8. **Comprobaciones posteriores.** Los validators que lo pidieron vuelven a decidir; en Aave, el factor de salud debe quedar en 1,2 o más.

V5 — permisos por agente

AJUSTE	V5	V4
Caducidad	Obligatoria, como mucho 365 días; `renewAgent` la amplía	0 significaba nunca
Límites nativos y de token (por lote, diario, total, periodo)	0 = nada permitido; «sin límite» solo como máximo explícito	0 significaba sin límite
Tokens permitidos	Hasta 50, cada uno con sus límites	Hasta 20
Destinatarios	Lista vacía = nadie	Lista vacía = cualquiera
Call policies	Por agente; validator obligatorio en toda llamada a protocolo; anclas extra para tokens, destinatarios y conjuntos de valores	Por wallet; anclas por posición, validator solo para dos funciones
Pausa	Por agente, más una pausa global	Solo global
Bloquear un protocolo	El dueño puede bloquear un destino para todos los agentes a la vez	—

V5 — la parte del dueño

- **Crear la wallet exige la passkey.** `createWallet` verifica una firma de la passkey sobre la dirección de la wallet y la credencial, así que nadie puede desplegar la wallet de otro ni elegir sus guardianes (la clase de BVCC-04 y BVCC-06, cerrada para siempre). La firma no lleva chain id a propósito, para que la misma wallet se pueda desplegar en todas las redes.
- **La credencial vive en la wallet** y solo cambia al crearla, por el dueño o por una recuperación, que pone la nueva (BVCC-07).
- **Las firmas del dueño caducan.** Toda UserOperation lleva un `validUntil` que firma la passkey y aplica el EntryPoint; la app usa unos 15 minutos.
- **Recuperación por votos** sobre una propuesta concreta, como se describe en BVCC-09. Ejecutarla pausa a todos los agentes.
- **La ejecución del dueño** mantiene los tres casos de comisión de V4 (nativo, transferencia ERC-20, otras llamadas) al 0,05 %, o al 0,15 % en una wallet de agentes, con semántica SafeERC20 y lecturas de saldo limitadas a 32 bytes de respuesta.
- **Reentrada:** se mantienen las dos capas de V4 (la comprobación de EOA en cada llamada y ningún `execute()` externo durante un lote de agente), ahora en almacenamiento transitorio.

V5 — validators

La interfaz devuelve cinco valores: sí o no, los tokens a medir, los costes, si quiere comprobación posterior y si se cobran las entradas. Los validators son contratos `view` que se llaman con `staticcall`; un revert significa no.

VALIDATOR	QUÉ DEJA HACER A LOS AGENTES Y CÓMO LO LIMITA
Universal Router	Swaps de token a token (v3 y v4). Bytes de comando exactos; todo destinatario es la wallet; cada salto v4 decodificado, cada hook ninguno o aprobado; todas las monedas

	de la ruta declaradas; valor nativo rechazado.
PositionManager de Uniswap v4	Crear, añadir, retirar, quemar, cobrar. Solo posiciones de la wallet; añadir exige que no haya aprobación individual; nada de sacar y meter en una llamada; hooks ninguno o aprobado en cada acción; las dos monedas del pool declaradas; entradas cobradas.
Pool de Aave v3	Depositar, retirar, pedir prestado (variable), devolver, ajustes de colateral y eMode. Beneficiario fijado a la wallet; un préstamo cuenta como coste contra el presupuesto; las retiradas se cobran como entrada; factor de salud ≥ 1,2 tras cualquier cambio con riesgo.
Posiciones de Uniswap v3	Crear hacia la wallet; añadir solo en posiciones propias sin aprobación; retirar, quemar y cobrar solo posiciones propias, cobrar exactamente hacia la wallet. Todo lo demás, y cualquier valor nativo, rechazado.
Nativo envuelto	Solo `deposit()` con valor y `withdraw(importe)` sin él. El valor ya se cobra en los límites nativos.

Clones y librería: cada wallet es un clon ERC-1167 de un contrato de lógica con los inicializadores desactivados; el código de permisos de agentes es una librería externa a la que se llega por DELEGATECALL, sin almacenamiento propio. La dirección del registry es una constante dentro de cada wallet.

V3 y V4 — call policies y registry de validators

Modelo de ejecución del agente

El agente llama a `executeAsAgent(bytes32 mode, bytes executionData)` como una transacción EOA ordinaria (no una UserOp). `mode` es el selector de batch ERC-7579 (`0x0100…00`); `executionData` es un `Execution[]` codificado en ABI de `(target, value, callData)`. El orden de validación dentro de la llamada es **per-tx → destinatario/protocolo → período → diario → total**, con el estado actualizado **antes** de la ejecución externa (checks-effects-interactions) y un guard `nonReentrant`.

ALTA (exfiltración de fondos por el agente) CORREGIDO — V3 DESPLEGADO 2026-07

Descripción

Hasta V2, el presupuesto del agente se descontaba cuando el valor salía de la wallet por una vía que la wallet podía ver: una transferencia nativa, un `transfer` ERC-20 o un `approve`. Una llamada a protocolo (caso 3) se permitía en cuanto su destino estaba en `allowedProtocols`. Eso no basta, porque varios puntos de entrada DeFi reciben el destino como argumento:

- `Pool.withdraw(asset, amount, to)` de Aave envía la posición suplida directamente a `to` — sin `approve` y sin `transfer`, así que el presupuesto no se enteraba.
- Las llamadas de swap y LP (`exactInputSingle`, `execute` del Universal Router, `mint`) llevan un argumento `recipient` que el agente elegía libremente.

Una clave de agente robada o defectuosa podía por tanto poner su propia dirección como destino y vaciar todo lo que la wallet hubiera suplido o aprobado a un protocolo whitelisteado **sin consumir un solo wei de su presupuesto**. La severidad es ALTA: la pérdida solo está acotada por la exposición DeFi de la wallet, y la contabilidad on-chain no muestra nada.

Remediación (V3) — call policies por selector

Las llamadas de caso 3 son ahora **default-deny por selector**. Whitelistar el protocolo ya no basta: el propietario debe registrar además una policy que diga *a dónde puede ir el dinero*. Las policies se escriben con `setCallPolicy(target, selector, policy)`, que solo puede llamar la propia wallet — en la práctica el passkey del propietario, incluido en la misma firma que autoriza al agente. La policy es una única palabra `uint256`:

BITS	CAMPO	SIGNIFICADO

255	POLICY_ALLOWED	El selector está permitido. Sin él → SelectorNotAllowed
254	POLICY_DEEP	Deriva la llamada al registry de validators on-chain (ver Anexo A)
223..192	PIN_WALLET (bitmap 32 bits)	La palabra *i* del calldata debe ser address(this)
191..160	PIN_PROTOCOL (bitmap 32 bits)	La palabra *i* debe ser una dirección de allowedProtocols

La comprobación dentro de executeAsAgent ocurre antes de cualquier llamada externa: (1) se lee el selector del calldata (< 4 bytes → CalldataTooShort); (2) la policy debe llevar POLICY_ALLOWED; (3) cada palabra anclada se compara **a palabra completa**, lo que además rechaza bits altos sucios, y una palabra leída más allá del final del calldata vale cero y por tanto falla — la comprobación es fail-closed por construcción; (4) si está POLICY_DEEP, el registry debe devolver true. Se añadieron cuatro custom errors: SelectorNotAllowed, PinnedArgMismatch, CalldataTooShort y PolicyValidationFailed. También cambió un default relacionado: una lista allowedProtocols vacía ahora revierte NoProtocolsWhitelisted en lugar de significar «cualquier protocolo».

> **El propietario nunca está sujeto a las call policies.** Solo aplican a los agentes. Una persona que firma con su passkey sigue usando cualquier dApp con normalidad — V3 estrecha la vía delegada, no la del propietario.

Policies registradas (se muestra Arbitrum One; las otras cinco redes usan las mismas formas)

PROTOCOLO	SELECTORES	ANCLAJE
Uniswap SwapRouter02	exactInputSingle · exactOutputSingle	PIN_WALLET palabra 3
Uniswap SwapRouter02	exactInput · exactOutput (multi-hop)	PIN_WALLET palabra 2
Aave v3 Pool	supply · withdraw · borrow · repay	PIN_WALLET palabra 2 / 2 / 4 / 3
Aave v3 Pool	repayWithATokens · setUserUseReserveAsCollateral · setUserEMode	Permitido sin anclaje — no existe argumento de destino
Permit2	approve(address,address,uint160,uint48)	PIN_PROTOCOL palabra 1 (el spender)
Uniswap Universal Router	execute(bytes,bytes[],uint256)	DEEP → registry de validators
Uniswap v3 NFPM	mint · collect	PIN_WALLET palabra 9 / 1
Uniswap v3 NFPM	decreaseLiquidity · burn	Permitido sin anclaje — el NFPM los limita al propietario del tokenId
WETH	deposit · withdraw	Permitido sin anclaje — ambos acreditan/pagan a msg.sender

> **Dos trampas de calldata encontradas al escribir los presets.**

- **Desplazamiento del offset.** El struct de `exactInput` (multi-hop) lleva un `bytes path`, lo que vuelve dinámica la tupla y mete una palabra de offset por delante: el destinatario está en la palabra 2, no en la 3 como en las variantes `*Single`. Anclar ahí la palabra 3 habría anclado `amountIn` y habría dejado el destino libre. Todos los offsets se re-derivaron codificando calldata real.
- **`increaseLiquidity` queda deliberadamente fuera de la whitelist.** A diferencia de `decreaseLiquidity`, `collect` y `burn`, el NFPM de v3 no le aplica ninguna comprobación de propiedad — solo el deadline. Un agente habría podido empujar los tokens que la wallet tiene aprobados al NFPM hacia un `tokenId` propiedad del atacante: un bypass de la lista de destinatarios que ningún anclaje de bitmap puede expresar, porque la palabra 0 es un `tokenId`, no una dirección. Los agentes añaden liquidez minteando una posición nueva con el destinatario anclado a la wallet; reactivar `increaseLiquidity` exigiría un validator profundo que compruebe `ownerOf(tokenId) == wallet`.

Qué cubre cada capa

CASO	OPERACIÓN	GOBERNADO POR
1	Transferencia de ETH nativo	`allowedRecipients` + límites nativos
2	`transfer(to, amount)`	`allowedRecipients` + límites de token
2b	`approve(spender, amount)`	`allowedRecipients` + límites de token
3	Llamada DeFi / a protocolo	`allowedProtocols` + **call policies**

Nota de alcance. V3 cierra la exfiltración de fondos por swaps, LP y retiradas de protocolo: el caso 3 es ahora seguro por construcción, configure lo que configure el propietario. Lo que no hace es convertir a un agente comprometido en inofensivo — un `transfer` o un `approve` directos (casos 2 y 2b) siguen gobernados por la lista de destinatarios y los límites por token, y esos los elige el propietario. Simulado en mainnet contra el agente de pruebas, con la lista de destinatarios vacía y los límites a cero, un `USDC.transfer` directo a un atacante pasa; con destinatarios y límites configurados revierte `RecipientNotAllowed`. Los límites on-chain son el modelo de seguridad.

Por qué hace falta un segundo mecanismo

Un anclaje de bitmap solo alcanza una palabra en un offset fijo. En el `execute(bytes commands, bytes[] inputs, uint256 deadline)` del Universal Router — y en el `modifyLiquidities` de Uniswap v4 — el destinatario va enterrado dentro de datos de longitud variable, así que no hay offset fijo. Esos selectores llevan en su lugar el bit `DEEP`, que delega la decisión en un validator on-chain sin estado.

Diseño

- **Dirección de despacho fija.** `BVCCValidatorRegistry` está compilado dentro de la wallet como constante (`0x5e371D54AC97a57B0a99145Ed04A3c9fA07850C2`). Como la palabra de policy solo lleva un flag y nunca una dirección, a un propietario no se le puede colar el validator de un atacante. Un test dedicado (`Create2Consistency.t.sol`) rompe la build si esa constante llega a desviarse de la dirección CREATE2 del registry.
- **Despacho fail-closed.** Sin validator registrado para el destino → `false`. Un validator que revierte, o que no tiene código, también deniega. Solo un `true` explícito deja pasar la llamada; cualquier otra cosa revierte `PolicyValidationFailed`.

- **Validators sin estado.** Son contratos `view` a los que se llega por `staticcall`. Nunca custodian fondos, nunca reciben approvals y no pueden mutar estado — solo responden sí o no.
- **Doble consentimiento.** Habilitar un protocolo complejo exige dos pasos independientes: BVCC registra el validator en el registry y el propietario registra la policy `ALLOWED|DEEP` con su passkey. Si falta cualquiera de los dos, la llamada se deniega.

Gobernanza asimétrica

DIRECCIÓN	FUNCIÓN	DEMORA
Permitir — registrar o sustituir un validator	`proposeValidator` → `activateValidator`	**Timelock de 48 h** + evento `ValidatorProposed`
Denegar — cancelar una propuesta pendiente	`cancelProposal`	Inmediata
Denegar — desactivar un validator activo	`freezeValidator`	Inmediata

La demora solo corre en la dirección permisiva, así que una propuesta maliciosa o equivocada es pública durante dos días antes de poder surtir efecto y los propietarios pueden reaccionar (pausar o revocar sus agentes). El peor caso con la clave de administración robada es por tanto una denegación de servicio o —tras 48 horas visibles públicamente— una degradación al nivel de protección de V2 en un protocolo. Nunca es una vía directa a los fondos: el registry no mueve nada.

El validator del Universal Router

- **Atado a un único router.** La dirección del router es un argumento inmutable del constructor; una llamada con cualquier otro destino devuelve `false`. Un router nuevo necesita su propio validator y su propio ciclo de gobernanza de 48 horas.
- **Coincidencia exacta del byte de comando** sobre `{0x00 V3_SWAP_EXACT_IN, 0x01 V3_SWAP_EXACT_OUT, 0x10 V4_SWAP}`. Cualquier otro byte se deniega, incluidas las variantes allow-revert y los comandos desconocidos.
- **Se comprueba cada destinatario**, incluida la acción `TAKE` de v4 (`{0x07 SWAP, 0x0b SETTLE, 0x0e TAKE}`): debe ser la wallet o el centinela `MSG_SENDER`. `SWEEP`, `TRANSFER`, `PAY_PORTION`, `WRAP` y `UNWRAP` se deniegan de plano.
- **El input malformado deniega.** Un calldata indecodificable revierte dentro del validator, y el registry lo trata como denegación; una cadena `commands` vacía se rechaza explícitamente.
- **Alcance.** Swaps token→token. Las patas nativas que dependen del `ADDRESS_THIS` del router quedan deliberadamente fuera; los swaps nativos se ejecutan como batches propios de BVCC a través de WETH, de modo que el router nunca retiene fondos de la wallet a mitad de camino.

> **Bug crítico detectado en revisión — la máscara de comandos.** La primera versión del validator enmascaraba los bytes de comando con `& 0x3f`. El router real enmascara con `& 0x7f` (el bit `0x80` es `FLAG_ALLOW_REVERT`), lo que significa que `0x40` **no** es un alias de `0x00`: es `ACROSS_V4_DEPOSIT_V3`, un depósito en un puente cross-chain. Con la máscara equivocada el validator lo habría aceptado como un swap v3 mientras el router bridgeaba los fondos de la wallet hacia un atacante en otra cadena: exactamente la clase de exfiltración que V3 existe para impedir. La corrección sustituyó la máscara por coincidencia exacta, ató el validator a un único router y rechazó las cadenas de comandos vacías. El comportamiento se fuzzea ahora sobre los 256 bytes de comando contra un modelo del router, y se volvió a confirmar en mainnet tras el despliegue (ver Anexo B).

Aave no necesita validator

Los argumentos de destino de Aave (`onBehalfOf`, `to`) están en offsets fijos, así que se resuelven con anclajes de bitmap dentro de la propia wallet. Ninguna policy de Aave lleva el bit `DEEP`, el registry no se consulta nunca para Aave y no interviene ningún timelock. Solo necesitarían validator las operaciones cuya seguridad no se puede expresar como «la palabra *N* debe ser la wallet» — flash loans, multicall, rutas por adaptadores — y esas quedan fuera de alcance.

Validators registrados — completo en todas las redes

Exactamente **dos** selectores llevan el bit `DEEP` y por tanto consultan el registry: `execute(bytes,bytes[],uint256)` del Universal Router y `modifyLiquidities(bytes,uint256)` del PositionManager v4 de Uniswap. Los dos están registrados y activos en las seis redes, leídos de `validators(address)` en el registry:

RED	VALIDATOR DEL UNIVERSALROUTER	VALIDATOR DEL POSITIONMANAGER V4
Arbitrum One	`0xD1522594e185C882bfDEc6FFD4DE8a53F69AF0Ca`	`0x6aE9c36121D5cD2d4Bde816F7F857AfB36Ccb8Eb`
Ethereum	`0x3615a0Fc0663C0201B543deC9816d5Ef30a82ffb`	`0xeFd47Aeb41e532D6a4ed04553b25827d88a4364d`
BNB Chain	`0x6945f861e0D7FCD566739000f3fDC1D44896D815`	`0x5f5Ee1068c2c54bBE896A08444958a289407Cd35`
Polygon	`0xC1C584E4149eD2EFe20DA0155b1B8257232102fF`	`0x62Eb47697BcF63dC9450B662d259AFf555958e25`
Base	`0x4A2Fc73AAdEfC84513defaa0c444d2150AC6A6D8`	`0xa2F35323DAcBFC2b5909B15607b511c6953e78a5`
Arbitrum Sepolia	`0x87550034627966e32d82E3b8AdB3f19c62E8d86E`	`0x32aA1B3E35dc6fF09EB7CDc8F6b43F3573AD2182`

Cada validator de PositionManager se comprobó uno a uno contra el PositionManager v4 al que está atado por su inmutable `POSITION_MANAGER`, y todos resuelven a la dirección correcta de su cadena. Los seis comparten el mismo `HOOK_REGISTRY` (`0x551C6e7ABdA04a110790888e711198f25621b066`, CREATE2, con código presente en cada una). Un atado cruzado habría fallado en cerrado y denegado en silencio toda la operativa de liquidez; de ahí que se verifique por red en lugar de deducirlo del script de despliegue.

> El `NonfungiblePositionManager` de Uniswap **v3 no** tiene validator registrado en ninguna red, y eso es correcto, no es un hueco. Sus policies son anclajes de bitmap — `mint` ancla el destinatario en la palabra 9, `collect` en la 1, y `decreaseLiquidity` y `burn` los controla el propio NFPM por dueño —, así que ninguna policy de v3 lleva el bit `DEEP` y el registry no se consulta jamás. `BVCCPositionManagerValidator` es un contrato de v4: está atado a un único PositionManager v4 e interpreta bytes de acción de v4. Registrarlo contra el NFPM de v3 rompería la liquidez v3 en lugar de asegurarla. `increaseLiquidity` sigue deliberadamente sin registrar por lo dicho en Anexo A: el NFPM no comprueba el dueño en esa función.

B Evidencia de pruebas

Las baterías de pruebas detrás de los hallazgos y sus remediaciones. Cada hallazgo del §4 tiene su test de regresión. Primero va V5.

Suite de tests de V5 — 191 tests: 145 locales y 46 sobre forks de redes

SUITE	TESTS	FOCO
`AgentWalletV5.t.sol`	58	Permisos del agente, medición, comisiones, restauración de aprobaciones, policies, pausa, caducidad
`SmartWalletV5.t.sol`	19	Ejecución del dueño, los tres casos de comisión, SafeERC20, creación con firma de la credencial
`ValidatorsV5.t.sol`	18	Validators del Universal Router, PositionManager v4 y Aave, registry y hook registry
`RecoveryV5.t.sol`	17	Votos, armado, retirada, cancelación, ejecución y pausa de agentes
`AgentInvariantsV5.t.sol` · `RecoveryInvariantsV5.t.sol`	11	Invariantes con 128.000 acciones aleatorias cada uno: límites, tokens permitidos, salida del dueño, aprobaciones del agente, contadores, atomicidad; armado y plazos de la recuperación
`SignaturesV5.t.sol`	7	ERC-7739 en clones, caducidad de firmas, reutilización entre wallets y entre redes
`GasV5.t.sol` · `DeployV5.t.sol`	11	Límites de gas; direcciones CREATE2 y conexiones
`RedTeamV5.t.sol` · `AuditRound2PoCsV5.t.sol`	4	Pruebas de concepto del equipo rojo y de la ronda 2, ahora tests de regresión
`fork/AllChainsDeployV5.t.sol`	6	Despliegue de producción en las seis redes con los monederos reales; operaciones del dueño y del agente por el EntryPoint real
`fork/ArbitrumForkV5.t.sol` · `EthereumUsdtForkV5.t.sol`	12	Universal Router, Permit2, Aave v3 y USDT reales
`fork/Audit*ForkV5.t.sol` · `RedTeamForkV5.t.sol`	13	Las pruebas de concepto de las rondas 2 a 4 y del equipo rojo, sobre contratos reales de Uniswap en cinco redes
`fork/V3PositionManagerForkV5.t.sol` · `WrappedNativeForkV5.t.sol`	15	Los dos validators añadidos tras el congelado

Los tests sobre fork simulan en local con el estado real de la red; no envían transacciones. La suite se ejecutó también con los tests de ataque de cada ronda: cada prueba de concepto fallaba con el código contra el que se escribió y pasa como regresión con el código congelado. Otros 13 tests adversariales de la revisión de los dos validators nuevos también pasan, incluidos WBNB en BNB Chain y WPOL en Polygon.

LOTE DE AGENTE (V5)	GAS
Solo nativo, o un token	~142.000 – 148.000
Cuatro tokens medidos	~163.000
Veinte tokens	~306.000
Peor caso, 60 tokens medidos	~956.000

El coste depende de los tokens que tenga permitidos ese agente, no del máximo. Crear una wallet V5 cuesta unas 537.000 unidades de gas. Las ejecuciones de V5 en redes reales están en el §5.

Baterías de V1 a V4

Ejecutada contra la wallet parcheada `0x1ED261…F0C3` con el EOA del agente `0x3852…4aB4`. Reverts confirmados por simulación; casos válidos como transacciones reales. Los destinatarios permitidos fueron los dos guardianes de recovery (controlados por el bot, fondos recuperables).

#	PRUEBA	ESPERADO	RESULTADO
1	ETH per-tx (0,0002 > máx 0,0001)	revierte	ExceedsPerTxLimit
2	ETH dentro de límite (0,0001)	pasa	Success
3	USDC per-tx (2 > máx 1)	revierte	ExceedsTokenMaxAmount
4	USDC diario (1+1+1, cap 2) — txs reales	la 3ª revierte	TokenDailyLimitExceeded
5	Envío de token a destino no permitido	revierte	RecipientNotAllowed
6	Envío de ETH a destino no permitido	revierte	RecipientNotAllowed
7	Presupuesto total (0,0002+0,0001 = 0,0003) — txs reales	la siguiente revierte	AgentBudgetExceeded
8	Presupuesto período (0,0003+0,0002 = 0,0005) — txs reales	la siguiente revierte	PeriodBudgetExceeded
9	Auto-rollover del período (periodStart 0)	la 1ª tx resetea la ventana	Verificado
10	Pausado (el owner pausa)	agente bloqueado	EnforcedPause
11	Expiración (caducó sola)	agente bloqueado	AgentPermissionsExpired

> **Observación.** La lista blanca de destinatarios aplica también a los **transfers de token**, no solo a ETH — un envío de token a una dirección no permitida revierte `RecipientNotAllowed`. La re-autorización vía “Edit” no resetea el contador diario (indexado por día UTC); con `periodStart = 0` la primera operación rota la ventana del período.

Resistencia a escalada de privilegios (de la sesión del 06-06)

Un agente que apunta a la propia wallet para llamar a funciones de owner (`pauseAgents`, `revokeAgent`, `authorizeAgent`) revierte con `AgentCannotCallWallet`; las llamadas arbitrarias a contratos fuera de la lista de protocolos están bloqueadas. El agente solo puede mover ETH y tokens permitidos hacia destinatarios permitidos.

Batería de pruebas — `forge test`: 264 pasan, 0 fallan

SUITE	TESTS	FOCO

`BVCCAgentWallet.t.sol`	50	Núcleo de permisos del agente: límites, presupuestos, expiración, pausa, batches
`UniversalRouterValidator.t.sol`	42	Corpus del router, destinatarios atacantes, fuzz sobre los 256 bytes de comando
`BVCCAgentWalletAdvanced.t.sol`	40	Contabilidad de comisiones, regresiones de `approve`, casos límite
`CallPolicyAdversarial.t.sol`	34	Intentos de bypass de policies, gobernanza del registry, calldata malformado
`BVCCPositionManagerValidator.t.sol`	26	Validación del calldata de LP en Uniswap v4
`BVCCWallet.t.sol`	23	Smart account base: ejecución, comisiones, recuperación por guardianes
`BVCCWalletFactory.t.sol` · `BVCCAgentWalletFactory.t.sol`	18	Factories CREATE2, direcciones deterministas, kill switch
`BVCCHookRegistry.t.sol`	11	Registry de aprobación de hooks y su timelock
`AaveIntegration.t.sol`	10	Ciclo de Aave sobre fork de mainnet y variantes adversariales
`BVCCPMValidatorSdkVectors.t.sol` · `ERC7739*.t.sol` · `Create2Consistency.t.sol` · `GasBench.t.sol`	10	Vectores dorados del SDK, firmas ERC-7739, guarda de congelación de direcciones, gas

Tests adversariales de policies (`CallPolicyAdversarial.t.sol`): los `withdraw`/`supply`/`borrow`/`repay` de Aave con un `to` u `onBehalfOf` externo revierten `PinnedArgMismatch`; un selector no registrado revierte `SelectorNotAllowed`; `PIN_PROTOCOL` rechaza un spender no whitelisteado; una llamada DEEP deniega cuando el validator devuelve false, revierte, no tiene código o no está registrado, y solo pasa con `true`; el timelock de 48 h, el freeze y el cancel se comportan según lo especificado; `policy = 0` revoca; el calldata corto, los anclajes fuera de rango y los bits altos sucios se rechazan; y un batch con un solo elemento malo revierte entero.

Aave sobre fork de mainnet (`AaveIntegration.t.sol`): ciclo completo supply → borrow → repay → withdraw, con la comisión de agente del 0,15% cobrada exactamente en `withdraw` y `borrow` y no cobrada en `supply` ni `repay`; consumo de presupuesto verificado por la vía del `approve`; un beneficiario externo en cualquiera de las cuatro llamadas revierte; `flashLoanSimple` y un `setUserEMode` no registrado revierten; un Pool ausente de `allowedRecipients` revierte; y el batch approve+supply hace rollback atómico si falla.

La capa de permisos se había atacado desde todos los ángulos a lo largo de tres rondas; la recuperación por guardianes y la vía de firma, no. Las dos custodian fondos, así que las dos recibieron el mismo trato: veinte tests escritos para conseguir que funcione algo que no debe (`test/RecoveryAdversarial.t.sol`, `test/SignatureAdversarial.t.sol`).

14.1 Vía de firma — sin hallazgos

Diez ataques contra la vía ERC-7739 / WebAuthn, los diez rechazados. Los dos que más importan son los de replay, porque el diseño de esta wallet invita a ambos: la dirección es idéntica en todas las

redes, y una misma passkey respalda una smart wallet y una agent wallet en dos direcciones distintas.

ATAQUE	RESULTADO
Reutilizar en otra red una firma hecha para la misma dirección	**Rechazado** el chainId va atado en el dominio del verificador
Reutilizar una firma entre la smart wallet y la agent wallet del mismo dueño	**Rechazado** el contrato verificador va atado
Firma maleada (`s` sustituida por `N − s`)	**Rechazado**
Firma a cero	**Rechazado**
Firma sobre otro contenido, o sobre el dominio de otra dApp	**Rechazado**
`authenticatorData` manipulado (quitando el flag de verificación de usuario)	**Rechazado**
Challenge cambiado por otro digest	**Rechazado**
`webauthn.create` colado donde se exige `webauthn.get`	**Rechazado**

Verificación on-chain — Arbitrum One (mainnet)

Wallet de pruebas `0x605A4C65dBa64bd26d9F7701e1e4Cda48c594A5c`, agente `0x38529C66F3cf22453D66B9E2A20FdF2676544aB4`.

FECHA	OPERACIÓN	RESULTADO
2026-07-16	Swap del agente USDC→WETH por SwapRouter02, dirigido por el servidor MCP `0x4768603d38b03f9823d5d25e53fb03970b8e4d294ebee791d0ae9ef92e71a8ae`	**OK** USDC debitado exacto, WETH a la wallet, comisión 0,15% exacta
2026-07-20	Swap del agente por el Universal Router — batch atómico de 3 llamadas que atraviesa tres capas distintas `0x6f831b37ae5db240256bf6bf99c8ce418edb75a1c38e37d6ed3e2c3a052a3bc0`	**OK** gas 419.124 · USDC −0,100000 · comisión 0,000000078468831120 WETH = 0,15% exacto · todos los destinatarios, la wallet
2026-07-20	El mismo swap con `recipient` = atacante (simulado, sin mover fondos)	**DENEGADO** revierte antes de ejecutar

Las tres llamadas de ese batch atraviesan cada una un guardián distinto: `USDC.approve(Permit2)` es caso 2b y se comprueba contra `allowedRecipients`; `Permit2.approve` es caso 3 con `PIN_PROTOCOL` en el spender; `UniversalRouter.execute` es caso 3 con `DEEP` y lo decide el validator.

Comportamiento del gate, medido antes y después de la activación del validator (48 h):

`REGISTRY.VALIDATE(...)`	ANTES DE ACTIVAR	DESPUÉS DE ACTIVAR
Swap legítimo (recipient = wallet)	false	**true**
Intento de robo (recipient = atacante)	false	false
Comando `0x40` (puente cross-chain)	false	false
Destino ≠ el router al que está atado	false	false

La corrección de la máscara queda por tanto confirmada en mainnet, no solo en tests, y la propiedad fail-closed se ve en la columna «antes»: hasta que el validator no estuvo activo, incluso el swap

legítimo se denegaba.

Gas

OPERACIÓN	GAS
`createWallet`	`4.548.977`
`authorizeAgent`	`166.803`
`setCallPolicy`	`25.912`
`executeAsAgent` (caso 3, con policy comprobada)	`64.758`

C Análisis estático

Slither se pasó sobre el árbol en cada ronda. Este anexo recoge los resultados y, tan importante como eso, qué resultados son falsos positivos y por qué — para no volver a investigarlos, y para que nadie borre las dos funciones virtuales que marca como código muerto.

Análisis estático — V5

Slither 0.11.3 se pasó sobre el árbol de V5 el 2026-10-26, antes de las rondas de revisión. Encontró un problema real: una lectura de `allowance` que copiaba todos los datos que devolviera el token (un «return bomb», la misma clase que la lectura de `balanceOf` sin límite señalada en V4). Se corrigió; en V5 toda lectura de la respuesta de un token está limitada a 32 bytes y tiene tope de gas. El resto de resultados son por diseño (la hora del bloque para plazos y caducidades, una llamada al registry por cada operación de protocolo de un lote) o falsos positivos. Se aplicaron las lecciones de V4: la pragma es exacta y se conservan las dos funciones que Slither marca como no usadas.

Generaciones anteriores

Slither 0.11.3 se ejecutó sobre ambos repositorios antes del redespliegue. Reportó 39 resultados en 59 contratos; **ninguno accionable**. La tabla resume las categorías y su tratamiento.

CATEGORÍA	EJEMPLOS	TRATAMIENTO
Patrones por diseño	Comparaciones con `block.timestamp`, ensamblador, llamadas de bajo nivel, calls-en-bucle (batch)	Aceptado
Falsos positivos	Mapping “no inicializado”; reentrancy en `createWallet` (solo un `emit` tras un `setGuardians` de confianza)	Sin acción
Código muerto — `_feeNumerator()`	Marcado como no usado	Falso positivo — verificado en uso (ver nota)
Falta zero-check — `owner_` de la factory	El constructor asigna owner sin validar	Corregido
Cosmético	Nomenclatura, dígitos literales, complejidad ciclomática	Aceptado

> **Nota de verificación — un hallazgo de herramienta no es un hecho.** Slither marcó `_feeNumerator()` como código muerto eliminable. Una comprobación del código mostró que **sí** se usa para calcular la comisión (base 0,05% / override del agente 0,15%); eliminarlo habría roto la lógica de comisiones. El hallazgo era un falso positivo causado por el dispatch de funciones virtuales y **no** se actuó sobre él.

Remediación aplicada

Se añadió un control defensivo de dirección cero — `if (owner_ == address(0)) revert ZeroOwner();` — al constructor de ambas factories, sincronizado en los dos repositorios. Suite completa en verde: **forge test = 128/128** en cada repo. Slither se re-ejecutó sobre los contratos V2 tras el endurecimiento de gas (Anexo D): **0 resultados**; la suite unitaria queda en **131/131**.

Análisis estático — Slither re-ejecutado sobre V3

Slither 0.11.3 se volvió a pasar sobre el árbol V3 el 2026-07-27 (`slither . --exclude-dependencies`, 65 contratos, 100 detectores): **54 resultados — 3 High, 29 Low, 22 Informational**. Cada resultado de impacto alto se contrastó contra el código fuente antes de aceptarlo o descartarlo. Uno de ellos es real: el resultado `reentrancy-eth` es un hallazgo ALTO confirmado, reproducido con un test de prueba de concepto y todavía abierto a fecha de redacción (BVCC-01).

> **La pasada que cuenta es esta, no la de V2.** La de V2 devolvió «0 resultados», pero solo porque el analizador no pudo bajar a IR el `ReentrancyGuard` con transient storage de OpenZeppelin y abandonó (Anexo C). Aquel cero era la herramienta rindiéndose, no un aprobado. El árbol de V3 se analiza de principio a fin —los detectores sí llegan a `executeAsAgent`, como demuestra el resultado de reentrancy de abajo—, así que este es el dato de análisis estático que tiene peso, y sus clases de resultados encajan con los 39 de la pasada de V1.

DETECTOR	OBJETIVO	VEREDICTO
`reentrancy-eth` confianza: media	`executeAsAgent` escribe `_currentAgent = 0` después de `super.execute()`	**CONFIRMADO — ALTA** — reproducido con un test PoC (BVCC-01)
`uninitialized-state` ×2 confianza: alta	`_tokenTotalSpent`, `_tokenDailySpent`	**Falso positivo** — ambos son `mapping`, que no tienen inicializador y valen cero por defecto

D Previamente remediado — V1 y V2

Estos hallazgos se reprodujeron y cerraron en generaciones que V4 y V5 sustituyen. Se conservan con su evidencia porque las wallets creadas por aquellas factories siguen vivas hasta que sus dueños migren, y porque V4 hereda las remediaciones que describen.

ALTA CORREGIDO Y REDESPLEGADO

Descripción

Los presupuestos **diario** y **total** por token del agente podían saltarse por completo. La validación contabilizaba el gasto de `transfer`, pero cargaba `approve` solo contra el cap por transacción — no contra el diario ni el total, y sin restringir el spender. El agente podía por tanto `approve` un allowance (solo capeado por per-tx) y vaciarlo con un `transferFrom` externo, sin tocar nunca los contadores acumulados.

Prueba de concepto (ejecutada on-chain, presupuestos agotados a diario 2/2 · total 2/3)

PASO	ACCIÓN	RESULTADO
1	`executeAsAgent → USDC.approve(B, 1)`	Pasa — per-tx OK, diario ignorado
2	Como EOA: `USDC.transferFrom(wallet, B, 1)` (fuera de `executeAsAgent`)	1 USDC extraído; contadores sin cambios

Un `transfer` plano del mismo importe sí revirtió correctamente con `TokenDailyLimitExceeded`; `approve` no. Al no tener límite diario, el ciclo `approve(1) → transferFrom(1)` era repetible — vaciando la wallet más allá de los caps diario (2) y total (3). Los fondos de la PoC se devolvieron a la wallet.

Remediación

En el bucle de acumulación de `executeAsAgent` se extendió la condición con `|| _isApprove(batch[i].callData)` de modo que `_applyTokenSpend` carga los `approve` contra los mismos caps diario/total y actualiza los contadores. La elección es deliberadamente conservadora (un `approve` sobrescribe el allowance, por lo que puede sobre-contar — la dirección segura). Se añadieron seis tests de regresión; **5 fallan sin el fix, todos pasan con él.**

> **Superficie acotada.** Los tokens aceptan solo `transfer` y `approve` por selector; `transferFrom` e `increaseAllowance` están bloqueados. ETH no tiene primitiva de delegación, así que no se ve afectado. `approve` era el único vector — ahora cerrado y re-verificado tras el redespliegue (ver Anexo B).

Ambas factories cambiaron de bytecode (zero-check), produciendo nuevas direcciones deterministas CREATE2 en Arbitrum Sepolia. La configuración de red del frontend se actualizó en consecuencia.

CONTRATO	NUEVA DIRECCIÓN (PARCHEADA)	ANTERIOR (DEPRECADA)
AgentWalletFactory	`0xc87aa10747A92B472EF6B36e190B84c897a2953e`	`0xe7ad…7e04`
SmartWalletFactory	`0xa5290A51a73903176e09C864E1542a07da67BD12`	`0x6890…F58d`

Separación de privilegios

- **Deployer (sin control):** `0xA3e06B6443D911915bb4eD157832d43C25D6617D` — el `owner` de la factory se fija desde un argumento del constructor, no desde `msg.sender`, así que el deployer no tiene privilegios.
- **Admin / kill-switch:** `0x3145Bd5e2489d8bDdAb17F23F26F07Ac5aD55A4c` — inmutable; la única acción privilegiada es el `kill()` de un solo sentido.

Tamaños runtime dentro del límite EIP-170 (AgentWallet 21.169 B; AgentWalletFactory 23.803 B — 773 B de margen).

La wallet base admite recuperación social: un quórum de 2 de 3 guardianes puede rotar la passkey del propietario tras un timelock. Se probó de extremo a extremo en la wallet `0xEBc851bA9e8Ce32A2cb8eB99544F5F024f06E61d` y ya está **completada**.

Proceso

- **Inicio (2026-06-06).** Recovery propuesta hacia una nueva passkey de propietario; los guardianes 1 y 2 (`0xa337…5908`, `0x8e06…2CcC`) aprobaron — quórum 2/2.
- **Timelock.** `recoveryReadyAt` on-chain = 2026-06-08 17:24 UTC; durante toda la ventana el propietario conservó la capacidad de cancelar.
- **Ejecución (2026-06-08).** Transcurrido el timelock se llamó `executeRecovery()` desde la UI `/recover`. El `signer` on-chain rotó a la nueva passkey (X `0x303591…` / Y `0xb858ed…`), reemplazando al propietario anterior (X `0x070f7a…`).
- **Comprobación positiva.** El nuevo propietario firmó de inmediato una transferencia real de **19 USDC** con el nuevo Face ID — confirmada on-chain.

Prueba negativa — la passkey antigua queda anulada

Para confirmar que la rotación revocó realmente la clave anterior, se intentó una transferencia con la passkey **antigua**, cuya credencial WebAuthn sigue presente en el autenticador de pruebas. La firma se genera localmente, pero debe validarse on-chain contra el propietario actual.

#	ACCIÓN	ESPERADO	RESULTADO
1	Enviar 0.0001 ETH, UserOp firmado por la passkey ANTIGUA	rechazo en validación	FailedOp(0, AA24 signature error)
2	Efecto on-chain (saldos ETH / USDC y nonce)	sin cambios	Sin cambios

> **Resultado.** La recovery se completó con éxito: la propiedad pasó a la nueva passkey, la nueva clave opera la wallet y la antigua es rechazada criptográficamente por el EntryPoint durante la validación de firma — ya no puede mover fondos aunque su credencial local siga existiendo.

ALTA (disponibilidad / sobrepago) CORREGIDO — V2 DESPLEGADA 2026-06-11

Descripción

En Arbitrum mainnet, los swaps a través de la wallet fallaban en la estimación de gas. La lógica de comisiones del Caso 3 escanea el calldata buscando candidatos a token y sondea cada uno con `balanceOf`. Los precompiles del sistema de Arbitrum (`0x64–0x6f`) reportan bytecode (`0xfe`), por lo que el control por longitud de código no los filtraba — y llamarlos consume **todo el gas reenviado** en lugar de revertir barato. El estimador subía el límite de gas hasta ~32M para que la sonda “cupiera”; ese límite inflado es lo que la wallet habría prefinanciado (riesgo de sobrepago 64×) y, en la práctica, la operación fallaba.

	ANTES (BUG)	DESPUÉS (FIX V2)

Estimación de gas (frontend / bundler)	~32.000.000	~400.000
Efecto de un candidato quema-gas	quema todo el gas reenviado	capado a 100k, la ejecución continúa
Resultado	el swap falla (o sobrepago 64×)	swap normal, ~$0,02–0,05 en Arbitrum

Remediación (V2)

- **Sondas capadas en gas.** Una constante compartida `PROBE_GAS_CAP = 100_000` capa cada staticcall de `balanceOf` tanto en `_tryBalanceOf` (snapshot) como en `_collectFeesOnIncrease` (cobro de comisión). El peor caso de un escaneo completo queda acotado a ~2,2M de gas frente a los 32M anteriores; un ERC-20 real necesita mucho menos que el cap, así que la contabilidad de comisiones no se ve afectada.
- **El cobro de comisión nunca puede romper una llamada del usuario.** Si un token se comporta mal tras el swap, su comisión se omite en lugar de revertir la transacción del usuario.
- **Cota contable explícita.** `executeAsAgent` ahora exige `totalEth ≤ type(uint128).max` antes del cast, eliminando un caso límite de truncamiento silencioso para agentes configurados sin límites de ETH.

Verificación

- **forge test = 131/131** — tests de regresión añadidos: un candidato quema-gas no deja sin gas al swap; un `balanceOf` que revierte se filtra sin ser fatal; un token con retorno sobredimensionado se snapshotea igualmente.
- **Slither 0.11.3 re-ejecutado sobre V2: 0 resultados.** (Nota de herramienta: Slither no puede generar IR para los internos del `ReentrancyGuard` con transient storage de OpenZeppelin — limitación conocida del parser, no un hallazgo.)
- **Tamaños EIP-170:** AgentWallet 21.214 B; AgentWalletFactory 23.848 B (728 B de margen); SmartWallet 13.432 B; SmartWalletFactory 16.030 B.

Despliegue V2 — CREATE2 determinista, misma dirección en todas las redes

CONTRATO	DIRECCIÓN (IDÉNTICA EN TODAS LAS REDES)
SmartWalletFactory V2	`0x230b7010529AB6977Dd8581B3eF018ef865BdEf1`
AgentWalletFactory V2	`0x8D9e24022777173AD6336e00884b6C87c7EF054c`

Desplegadas el 2026-06-11 en Arbitrum One (42161), BNB Chain (56) y Arbitrum Sepolia (421614). Las wallets creadas por factories pre-V2 no reciben el fix — las wallets nuevas deben crearse a través de las factories V2.

Se corrigieron tres incidencias en la aplicación web de la wallet, sincronizadas con el repositorio público.

ÁREA	INCIDENCIA	CORRECCIÓN
`lib/wallet.ts`	La recuperación de credencial consultaba solo el evento de la factory base, por lo que reabrir una wallet *de agente* por dirección fallaba (“no hay wallet activa”).	Consultar también `agentFactory` / `AgentWalletCreated`.
Página de agentes	Los límites por token se mostraban solo como texto.	Leer `getTokenSpent` on-chain; pintar barras de progreso Diario/Total (verificado en vivo).
Flujo de deploy	El gas que pasaba la app infraestimaba el deploy de la agent wallet (~21 KB) en Arbitrum,	Delegar en la estimación de red de la wallet.

	provocando fallos al firmar.

E Cronología de la remediación

RONDA	FECHAS	FOCO	RESULTADO
V1	2026-06-06 → 06-08	Modelo de permisos del agente, primer análisis estático	BVCC-A1 encontrado, corregido y redesplegado; batería de permisos y recuperación por guardianes verificadas de extremo a extremo
V2	2026-06-11 → 06-12	Comportamiento del gas en mainnet	BVCC-A2 encontrado y corregido; despliegue multichain determinista en seis redes
V3	2026-07-21	Capa de call policies y registry de validators	Cerrada una vía de exfiltración de fondos; BVCC-02 y BVCC-12 cazados dentro del propio trabajo, antes de desplegar
V4	2026-07-27 → 07-28	Revisión posterior a la publicación, segunda pasada con otra herramienta	BVCC-01, BVCC-04 a BVCC-08 y BVCC-11 encontrados y corregidos; desplegado y verificado en seis redes
—	2026-07-28 → 07-29	Activación de validators	Capa de validación profunda completada en las seis redes; ciclo de liquidez v4 ejercitado en mainnet
V5	2026-10-23 → 10-26	Diseño y código	Especificación con siete garantías; núcleo escrito; 146 tests, con forks de despliegue en seis redes; return bomb de Slither corregido
V5, ronda 1	2026-10-27	Revisión completa con una herramienta de IA	BVCC-14 a BVCC-20 encontrados; validator obligatorio en toda llamada a protocolo; entradas cobradas
V5, rondas 2–3	2026-10-28	Comprobación de arreglos, rutas de Uniswap sobre forks	BVCC-21 a BVCC-23 encontrados y corregidos en los validators
V5, ronda 4 · equipo rojo	2026-10-29	Hooks; ataques desde dentro y desde fuera	BVCC-24 a BVCC-30: cinco corregidos; BVCC-25, BVCC-29 y BVCC-30 aceptados
V5, ronda 5	2026-10-29	Arreglos del equipo rojo, preparación del congelado	Sin hallazgos nuevos; código congelado (176 tests en ese momento)
—	2026-10-30 → 11-02	Despliegue	Arbitrum Sepolia y después las cinco mainnets; validators de Uniswap v3 y nativo envuelto añadidos y revisados (sin hallazgos)
—	2026-11-01 → 11-08	Activación de validators y pruebas	30 de 30 validators activos tras las esperas de 48 horas; batería completa del agente y una recuperación V5 en Arbitrum Sepolia
—	2026-11-11	Verificación y migración	72 de 72 contratos con el código verificado; una wallet V4 migrada a V5 en BNB Chain; esta edición

A continuación, el registro de despliegue de V3, la build congelada, la capa off-chain y las notas de migración.

Despliegue V3 — CREATE2 determinista

CONTRATO	DIRECCIÓN (IDÉNTICA EN LAS SEIS REDES)
SmartWalletFactory V3	`0xD42F61AA856A4f47885Ecd2D0ce119411d53C192`
AgentWalletFactory V3	`0xd866a7563cDaC9F71423be3332b62c329C676064`
ValidatorRegistry	`0x5e371D54AC97a57B0a99145Ed04A3c9fA07850C2`
HookRegistry	`0x551C6e7ABdA04a110790888e711198f25621b066`

Redes: Arbitrum One (42161), Ethereum (1), BNB Chain (56), Base (8453), Polygon (137) y Arbitrum Sepolia (421614). Los cuatro contratos se re-comprobaron el 2026-07-27 vía la API de los exploradores: presentes, byte a byte idénticos y con código fuente verificado en las seis cadenas.

Los validators del Universal Router están atados a un router cada uno, así que su dirección cambia por red:

RED	VALIDATOR DEL UR	ESTADO EN EL REGISTRY
Arbitrum One	`0xD1522594e185C882bfDEc6FFD4DE8a53F69AF0Ca`	Activo
Ethereum	`0x3615a0Fc0663C0201B543deC9816d5Ef30a82ffb`	Activo
BNB Chain	`0x6945f861e0D7FCD566739000f3fDC1D44896D815`	Activo
Base	`0x4A2Fc73AAdEfC84513defaa0c444d2150AC6A6D8`	Activo
Polygon	`0xC1C584E4149eD2EFe20DA0155b1B8257232102fF`	Activo
Arbitrum Sepolia	`0x87550034627966e32d82E3b8AdB3f19c62E8d86E`	Activo

Estado leído directamente de `validators(router)` en cada cadena, la última vez el 2026-07-28, tras activar Arbitrum Sepolia. Un validator desplegado pero sin activar es inerte en la dirección segura: las llamadas DEEP correspondientes fallan cerradas. Los manifiestos de procedencia por red (direcciones más codehashes del router y del validator) están en `contracts/deployments/`. Los validators del `PositionManager` de Uniswap v4 se activaron en las seis redes en la misma tanda — ver Anexo A para las direcciones y la comprobación de atado por red, y Anexo D para el ciclo de liquidez ejecutado contra ellos.

Build congelada

Unas direcciones CREATE2 deterministas exigen una build congelada: solc **0.8.36**, `via_ir`, `optimizer_runs = 50`, EVM `cancun`. Ese compilador se eligió deliberadamente para esquivar el bug del pipeline via-IR del rango 0.8.28–0.8.33. El tamaño runtime de la factory de agentes es de **24.283 bytes**, con 293 bytes de margen bajo el límite EIP-170 — cualquier adición futura debe medirse contra ese margen.

Capa off-chain — SDK y MCP 0.2.0

Las librerías cliente se hicieron conscientes de las policies para que el guardián on-chain no se descubra solo al emitir la transacción. `getCapabilities` ahora condiciona un swap a las policies registradas *y* al registry de validators, no solo al struct de permisos, y devuelve la versión de la wallet junto con las policies que falten; los cuatro errores de V3 se decodifican en mensajes accionables que distinguen «no hay validator activo» (no reintentar) de «el validator rechazó este calldata» (corregible). `setCallPolicy` deliberadamente **no** se expone en el SDK: las policies las escribe el passkey del propietario, nunca la clave de un agente. El módulo de préstamos Aave y sus planners viajan por el mismo catálogo, siempre con el beneficiario anclado a la wallet.

Migración

El bytecode de la wallet V3 difiere del de V2, así que las direcciones CREATE2 también: los usuarios existentes crean una wallet nueva y mueven sus fondos, igual que en la migración V1→V2, tras lo

cual se hace `kill()` de las factories V2. Las capacidades futuras — registrar más validators, añadir presets de protocolos — son neutrales respecto al bytecode y no obligarán a otra migración.

F Identificadores y direcciones

ELEMENTO	VALOR
Red	Arbitrum Sepolia (421614)
Wallet parcheada	`0x1ED261BA084c4E6117daf289f3ca233BBA3cF0C3`
Agente (rol B)	`0x38529C66F3cf22453D66B9E2A20FdF2676544aB4`
AgentWalletFactory (Sepolia, pre-V2)	`0xc87aa10747A92B472EF6B36e190B84c897a2953e`
SmartWalletFactory (Sepolia, pre-V2)	`0xa5290A51a73903176e09C864E1542a07da67BD12`
SmartWalletFactory V2 (todas las redes)	`0x230b7010529AB6977Dd8581B3eF018ef865BdEf1`
AgentWalletFactory V2 (todas las redes)	`0x8D9e24022777173AD6336e00884b6C87c7EF054c`
SmartWalletFactory V3 (todas las redes)	`0xD42F61AA856A4f47885Ecd2D0ce119411d53C192`
AgentWalletFactory V3 (todas las redes)	`0xd866a7563cDaC9F71423be3332b62c329C676064`
SmartWalletFactory V4 (todas las redes)	`0xfd105197109244483b5f870501326E6faec9F93c`
AgentWalletFactory V4 (todas las redes)	`0xf3A61F9d64d45362E149A111289546523BCd26a6`
SmartWalletFactory V5 (todas las redes)	`0x2672951DC1E3f865BA6Fc98e5509dc4B23B260a2`
AgentWalletFactory V5 (todas las redes)	`0xE36fb66c21C38536454C1a58fBC5EB1Cb455aD52`
Lógica SmartWallet V5 (todas las redes)	`0xc9dE89d54beB786bc916131CA6c8c5374507Bd15`
Lógica AgentWallet V5 (todas las redes)	`0x360f991B23a3D5202Dc422292697aB07dA279cFA`
Librería de lógica de agentes V5 (todas las redes)	`0x7504bf3c1230eF99eaEFAc03287c8F036CBb6D0E`
ValidatorRegistry V5 (todas las redes)	`0x7269334AE74DCE4e607eec36268383DD35f28350`
HookRegistry V5 (todas las redes)	`0xEaEaC7F6071f7F576Be2aD97898Dd0c0F51BE78A`
Validators V5	Por red, en el §5
Monedero de comisiones V5 (todas las redes, constante)	`0x3e3eb089169a7315a994947465ce5f5FC3A307D4`

Prueba de migración V5 (BNB Chain)	`0x566669C6cC8FDD33c08Ed56A51c23ec1972A4936`
ValidatorRegistry (todas las redes, V3 y V4)	`0x5e371D54AC97a57B0a99145Ed04A3c9fA07850C2`
HookRegistry (todas las redes, V3 y V4)	`0x551C6e7ABdA04a110790888e711198f25621b066`
Posición LP v4 de prueba (Arbitrum One)	`tokenId 190877 — minteada y quemada por un agente`
Wallet de pruebas V3 (Arbitrum One)	`0x605A4C65dBa64bd26d9F7701e1e4Cda48c594A5c`
Admin de gobernanza (hardware wallet)	`0x3145Bd5e2489d8bDdAb17F23F26F07Ac5aD55A4c`
Redes V2/V3/V4/V5	Arbitrum One · Base · BNB Chain · Ethereum · Polygon · Arbitrum Sepolia
Build V3/V4/V5	solc 0.8.36 · via-IR · optimizer 50 · EVM cancun (congelada para CREATE2); V5 sin hash de metadatos
Herramientas	Slither 0.11.3 · Foundry (forge 1.5.1) · viem 2.x

Confidencial — Block Venture Chain Capital. Preparado para revisión interna e inclusión en el repositorio. La batería de permisos V1 se ejecutó en una testnet pública (Arbitrum Sepolia); la capa de call policies de V3 y el ciclo de liquidez v4 se verificaron además en Arbitrum One mainnet. Las factories V2, V3 y V4 están desplegadas en Arbitrum One, Ethereum, BNB Chain, Base, Polygon y Arbitrum Sepolia, con los validators del Universal Router y del PositionManager v4 registrados en las seis. La V5 está desplegada en las mismas seis redes con sus propios registries y cinco validators por red, todos activos, y todos los contratos de V5 tienen el código verificado.

G Notas de proceso

Se recoge porque una revisión que solo enumera lo que encontró exagera su propia fiabilidad.

Junto a los hallazgos conviene anotar tres lecciones. La reentrada se descartó una vez, con un razonamiento plausible sobre EOAs, hasta que una prueba de concepto lo contradijo: las suposiciones sobre la forma de una dirección merecen un test, no un argumento. Los problemas más graves de aquí aparecieron al volver a mirar código que ya había pasado una revisión — la apropiación de guardianes vino de una pasada con otra herramienta, la rotación de credencial de un seguimiento de esa misma pasada, y dos más de atacar la recuperación de frente. Y la cifra del análisis estático se movió en uno durante todo el proceso: no subió mientras la reentrada fue explotable durante tres generaciones, ni bajó cuando el arreglo la cerró. Las herramientas encuentran patrones; los hallazgos de aquí salieron de leer el código y preguntarse qué haría un atacante con él. Los contratos siguen siendo de revisión interna y no están auditados externamente.

La V5 añadió tres más. **Los arreglos traen vecinos.** Las rondas 2, 3 y 4 y el equipo rojo encontraron cada uno un camino nuevo a un paso del arreglo anterior, todos en los validators de posiciones de Uniswap: comprobación de dueño al añadir pero no al retirar, comprobación de la posición de destino pero no de sus aprobaciones, comprobación de hook en los swaps pero no en las retiradas. Las reglas solo aguantaron cuando se hicieron simétricas en todas las acciones. Después de arreglar un validator conviene preguntarse «¿qué acción hermana no ha tocado este arreglo?». **Los saldos no lo ven todo.** Medir lo que sale cerró BVCC-03, pero el valor también puede moverse como liquidez entre NFTs, como colateral o a través de un hook, sin que salga ningún token de la wallet. Cada caso necesitó su propia regla, y los que no se pueden descartar quedan escritos como supuestos de confianza. **Primero la especificación.** Las rondas de V5 se contrastaron con siete garantías escritas, lo que convirtió las discusiones sobre intenciones en tests que pasan o fallan. Donde la especificación era ambigua (G-4 para pagos de protocolos, BVCC-16), el hallazgo obligó a decidir en lugar de debatir. La ronda 5 señaló además que su auditor no tenía copia del código de la ronda 4 para comparar; esa comparación la hizo el equipo, y el código congelado queda ligado ahora a un commit, sus hashes, sus tests y sus direcciones.